

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Escola Superior d'Enginyeries Industrial, Aeroespacial i Audiovisual de Terrassa

Universitat Politècnica de Catalunya

Escola Superior d'Enginyeries Industrial, Aeroespacial i Audiovisual de Terrassa

MSc in Aerospace Engineering

Computational Engineering

CSM / PROJECT: WING MODELLING

Alex Carmona Peña

Pau Tusell Tresserras

Autumn 2023

Contents

1 Introduction and methodology **1**
1.1 Wingbox modeling type description . . . 2
1.2 Whole wing structure type description . . . 2

2 Results **3**
2.1 Part I results: Wingbox . . . 3
2.1.1 Beam discretization of the wingbox . . . 3
2.1.2 Shell discretization of the wingbox . . . 10
2.2 Part II results: Whole wing structure . . . 16
2.2.1 Bending and torsion tests . . . 16
2.2.2 Wind tunnel test using direct solving of the equations . . . 19
2.2.3 Wind tunnel test using a model-order reduction strategy . . . 22

3 Conclusions **26**

4 Annex **28**
4.1 MAIN BEAM . . . 28
4.2 MAIN SHELL . . . 33
4.3 MAIN WING . . . 41
4.4 MAIN MODAL . . . 48
4.5 COMPUTE PRESSURE STATE . . . 48
4.6 COMPUTE KM BEAMS . . . 49
4.7 COMPUTE KM SHELLS . . . 51
4.8 COMPUTE STRESS SHELLS . . . 54

1 Introduction and methodology

The aim of this assignment is the numerical resolution of the static response and the extraction of the modes of a wing with a NACA 0012 airfoil under different loads using the finite element method with a code written in a C language (in Matlab to be more precise).

The problem is divided into two parts.

In the first part, only the wing box has been analyzed, first using a beam modelization for the wing box and then discretizing the true geometry of the wing box using shell elements. In this first part, the wing box is subjected to two different loads independently, a force and a torque.

In the second part, a more real structure has been assumed by considering the wing to include the wing box, stringers, ribs, and skin, where the stringers are modeled using beam elements and the rest of the components with shells. In part two, three cases are analyzed:

- The wing under the same two loads as in part one.
- The wing is analyzed under more real loads, corresponding to the conditions in a wind tunnel test. These are a pressure load (aerodynamic forces) combined with body forces (the weight of the wing components).

As for the solver, it should be noted that the finite element problem will be solved first, for all parts, through the direct solving of the system of equations. Finally the end of part II, a model-order reduction strategy based on a modal projection, selecting the most appropriate modes for the wind tunnel test problem, will be implemented and the solution will be compared with the direct solution.

All data is known from the mentioned NACA airfoil. A summary can be found in the table below:

Data	Value
$c\ (m)$	0.8
$b\ (m)$	0.55
$y_0\ (m)$	0.3083
$y_1\ (m)$	0.176
$y_2\ (m)$	0.464
$h_1\ (mm)$	17.5
$h_2\ (mm)$	15
$h_3\ (mm)$	6

Property	Value
Area (m^2)	0.0062
Inertia y (m^4)	$8.801 \cdot 10^{-6}$
Inertia z (m^4)	$8.069 \cdot 10^{-5}$
Polar inertia (m^4)	$8.949 \cdot 10^{-5}$
Shear correction y	0.5457
Shear correction z	0.1767
Torsion correction	0.2920
Shear center (m)	0.2954

Table 1: NACA 0012 airfoil geometry and properties

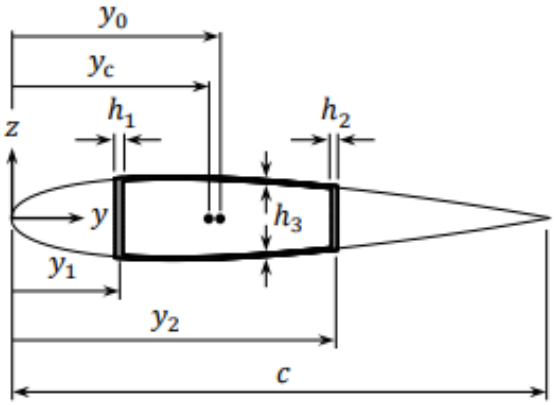


Figure 1: Representation of the NACA 0012 section

1.1 Wingbox modeling type description

One tip of the wingbox is supposed to be attached to the aircraft fuselage, hence a fixed condition has been imposed. A force and, later, a torsion have been applied to the other tip, moving the entire structure from its rest position. This allowed the analysis of deflections, twist, stress and modal analysis along the wingbox.

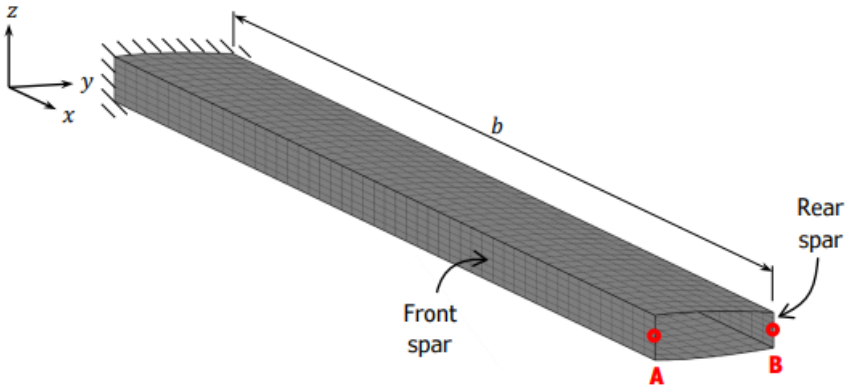


Figure 2: Representation of the wingbox

Notice the tip loads have been applied to A and B nodes for the shell case and at the last node for the beam one.

1.2 Whole wing structure type description

This realistic model has different elements resulting in a combination of beams and shells discretizations. The scheme of this complex model is the following:

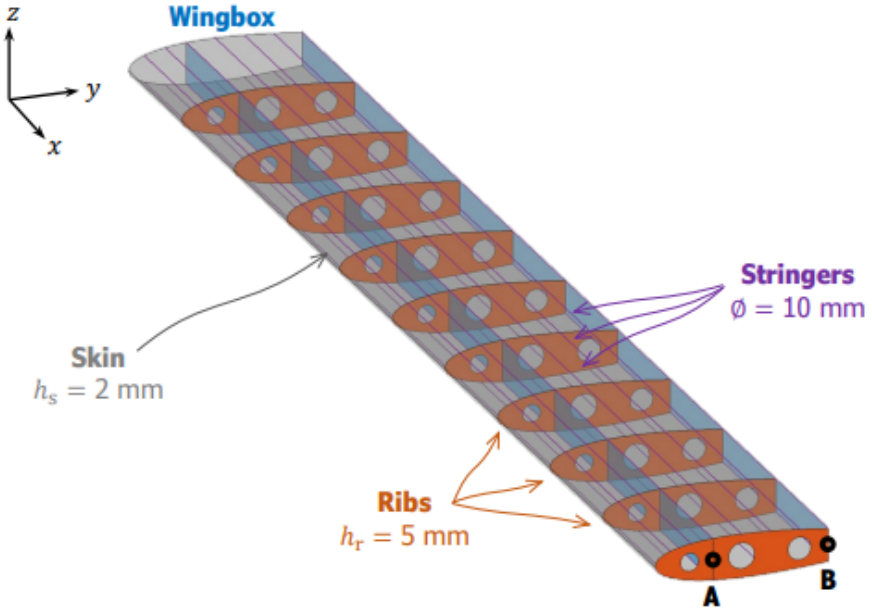


Figure 3: Representation of the wing

Like in the previous case, the two cases of unitary force and torque have been analyzed by applying the loads to the wing tip at nodes A and B.

After solving these elemental cases, another case has been performed by considering the weight of all the components of the wing and a pressure distribution over the skin surface. The pressure distribution on the skin at a wing section is shown in Figure 4.

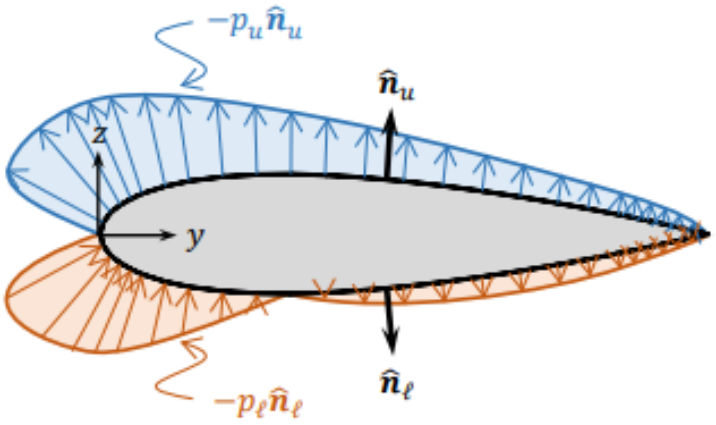


Figure 4: Representation of the pressure on the skin at a wing section

The equations that describe the pressure distribution are given below. Note that the pressure vector is obtained by multiplying by the normal vector at each node.

$$p_u(x,y) = -4\alpha P(x)\left[(1-\frac{y}{c})^4 + \sqrt{1-\frac{y}{c}}\right] \tag{1}$$

$$p_l(x,y) = 4\alpha P(x)\left[(1-\frac{y}{c})^4 - \frac{1}{4}\sqrt{1-\frac{y}{c}}\right] \tag{2}$$

$$P(x) = \begin{cases} p_\infty & for \;\; x \leq 0.8b \\ p_\infty \cdot cos\left(\frac{\pi}{2}\frac{x-0.8b}{0.2b}\right) & for \;\; x \geq 0.8b \end{cases} \tag{3}$$

With $\alpha = 10°$ and $p_\infty = 1.25 \cdot 10^4$ Pa.

2 Results

2.1 Part I results: Wingbox

2.1.1 Beam discretization of the wingbox

1. Vertical deflection

   The upward force applied to the wing tip results in an upper tendency along the wingbox. The obtained result is logical, but its shape (linear, quadratic...) was not so trivial: as can be checked below, at the beginning the deflection increases exponentially whereas after 3 meters the behaviour of the deflection is almost lineal.

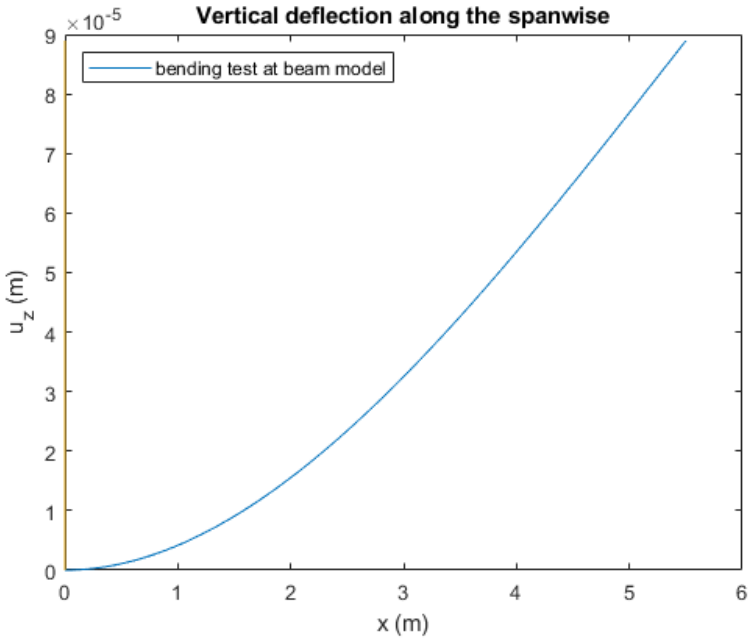


Figure 5: Vertical deflection distribution along the spanwise for the bending force and beam discretization

The same vertical deflection analysis has been done for the torsion case. In this occasion the torque applied to the tip doesn't generate any deflection over the z direction as the results state. Plus, the structure doesn't suffer any movement in the other directions due to the particular performed test.

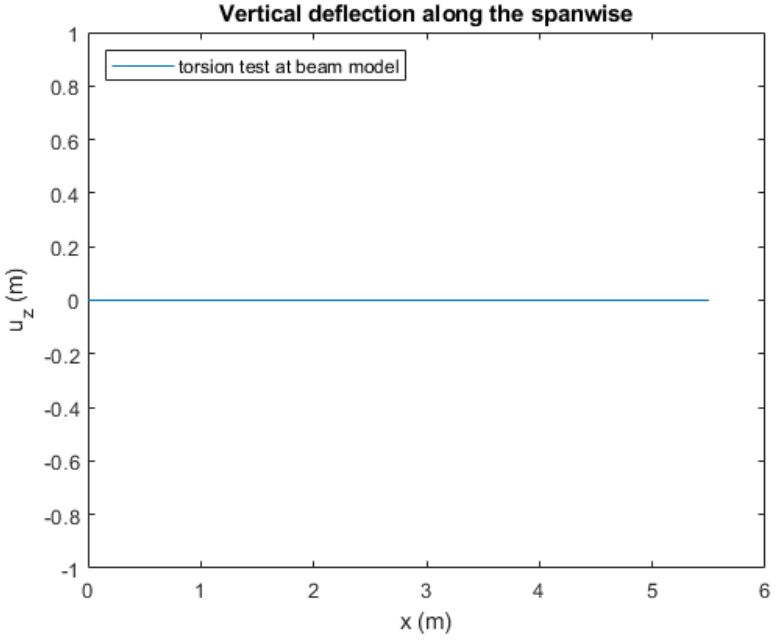


Figure 6: Vertical deflection distribution along the spanwise for the torsion force and beam discretization

2. Twist angle

   Yet again both bending and torsion tests have been considered, being the only difference on the result evaluation. In other words, in this occasion the twist angle of the wingbox has been the evaluated property. As expected it is zero along the entire structure for the bending test (see figure 7) as the force has been applied perfectly in a vertical direction, along the x axis to be precise. On the other hand, the torque applied to the tip generates a twist over the entire wingbox with a linear shape (see figure 8).

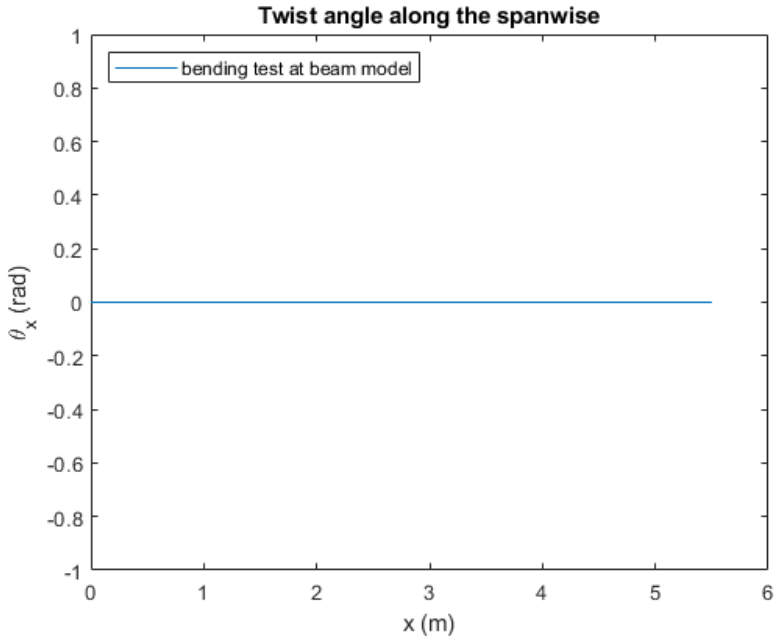


Figure 7: Twist angle distribution along the spanwise for the bending force and beam discretization

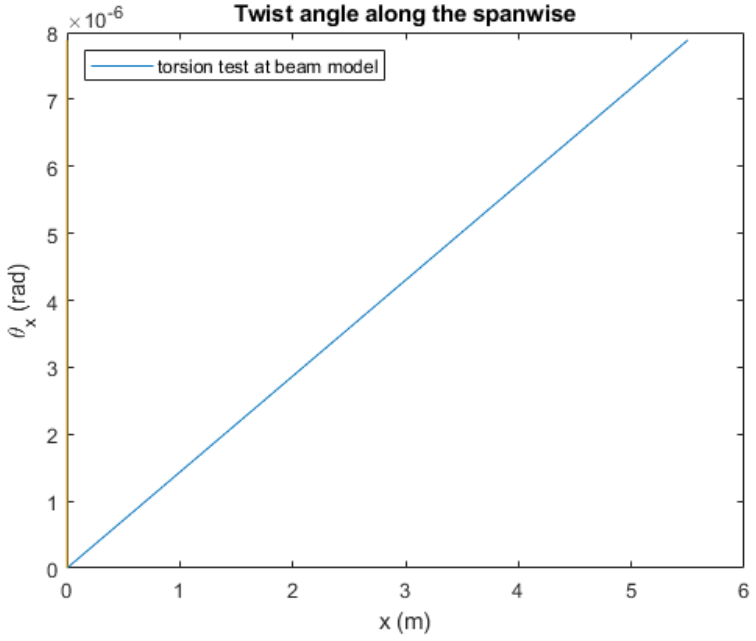


Figure 8: Twist angle distribution along the spanwise for the torsion force and beam discretization

3. Modal analysis

   Deflections over y and z directions as well as the twist over x have been computed for the first six natural modes. The corresponding frequencies are presented below as well as the pertinent plots (note all of them are presented in a new page for a better visualization).

f_1 (Hz)	f_2 (Hz)	f_3 (Hz)	f_4 (Hz)	f_5 (Hz)	f_6 (Hz)
3.87	11.66	23.98	66.10	70.67	83.68

Table 2: Frequencies of the first six natural modes of the wingbox modelled with beams

u_y modal analysis for beam discretization

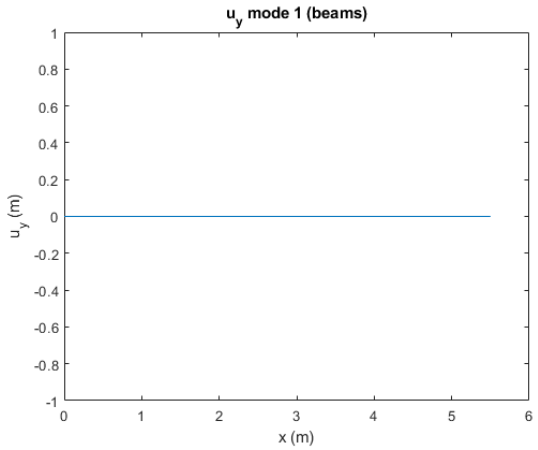


Figure 9: First mode deformed shape along y axis for beam discretization

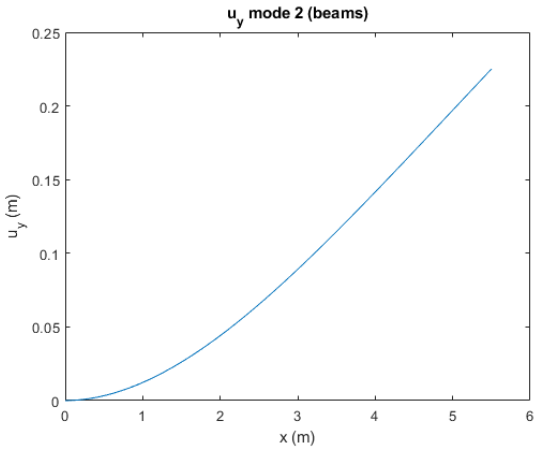


Figure 10: Second mode deformed shape along y axis for beam discretization

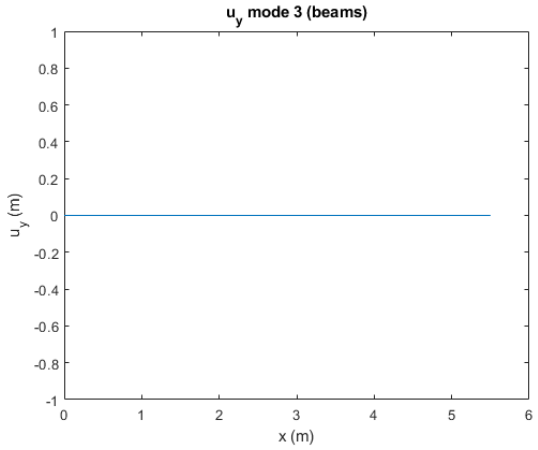


Figure 11: Third mode deformed shape along y axis for beam discretization

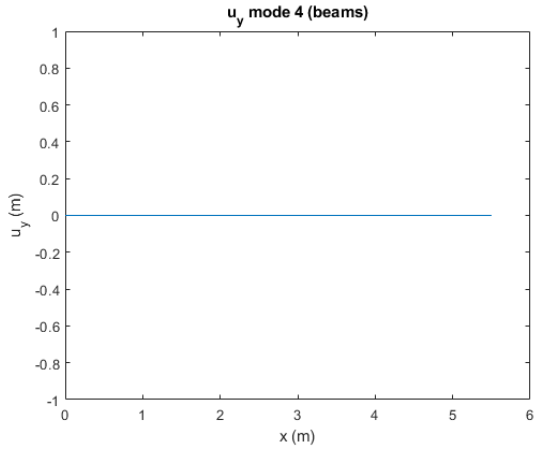


Figure 12: Fourth mode deformed shape along y axis for beam discretization

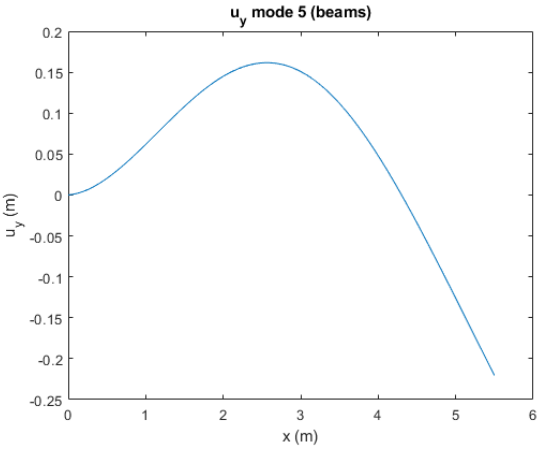


Figure 13: Fifth mode deformed shape along y axis for beam discretization

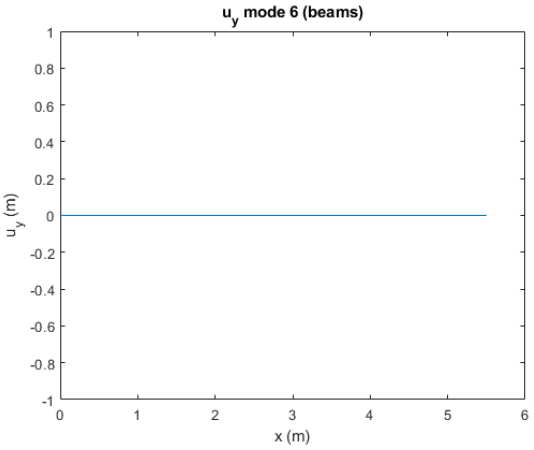


Figure 14: Sixth mode deformed shape along y axis for beam discretization

u_z modal analysis for beam discretization

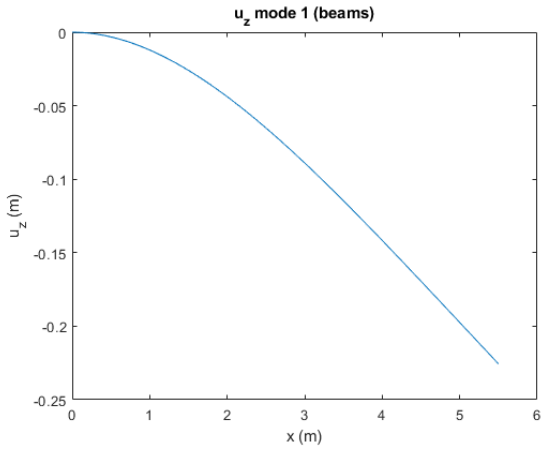


Figure 15: First mode deformed shape along z axis for beam discretization

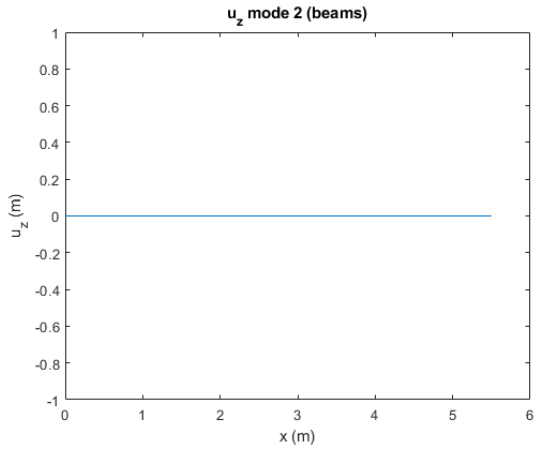


Figure 16: Second mode deformed shape along z axis for beam discretization

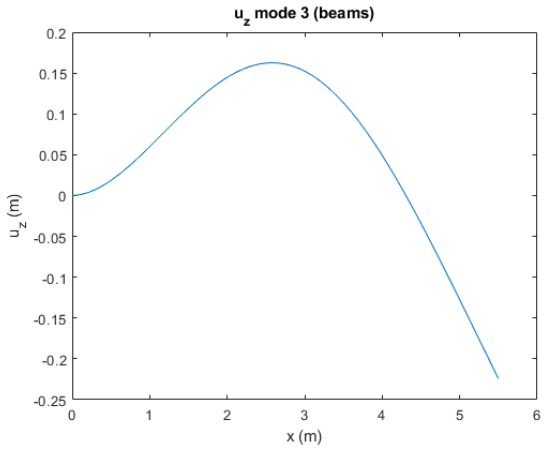


Figure 17: Third mode deformed shape along z axis for beam discretization

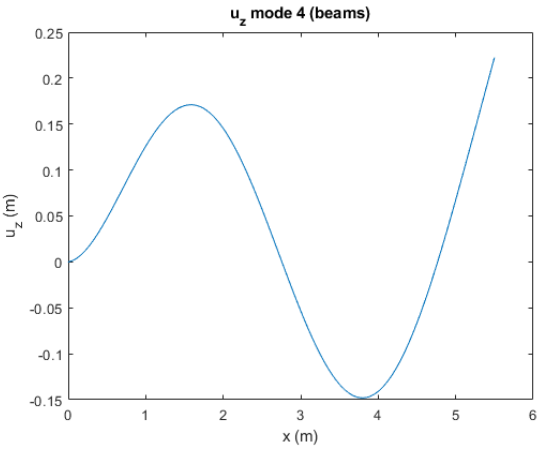


Figure 18: Fourth mode deformed shape along z axis for beam discretization

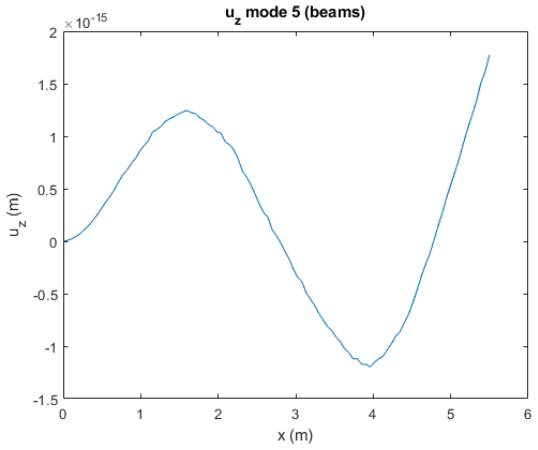


Figure 19: Fifth mode deformed shape along z axis for beam discretization

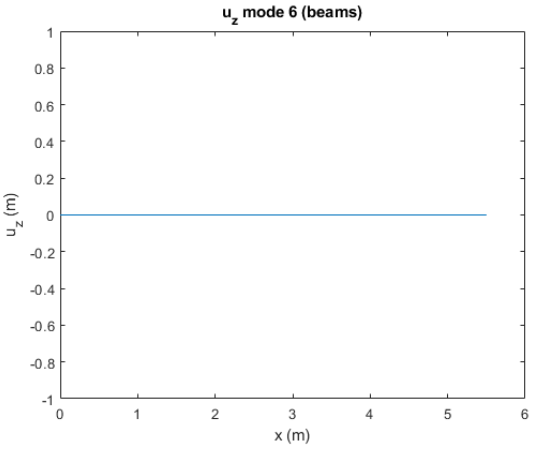


Figure 20: Sixth mode deformed shape along z axis for beam discretization

θ_x modal analysis for beam discretization

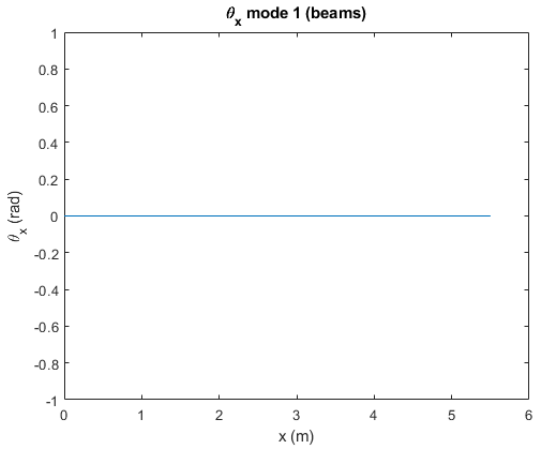


Figure 21: First mode twist angle shape along x axis for beam discretization

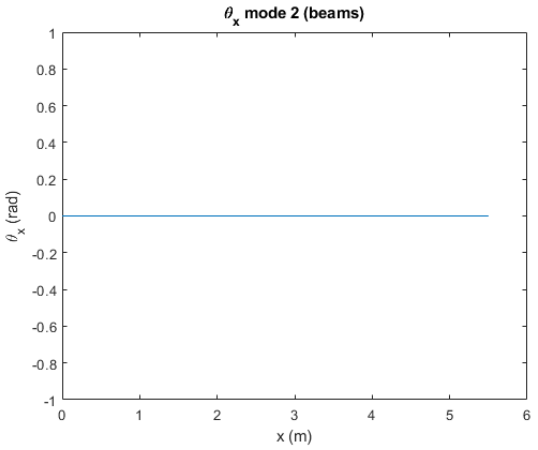


Figure 22: Second mode twist angle shape along x axis for beam discretization

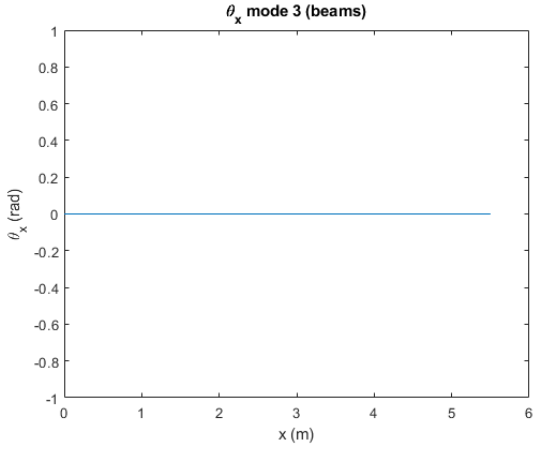


Figure 23: Third mode twist angle shape along x axis for beam discretization

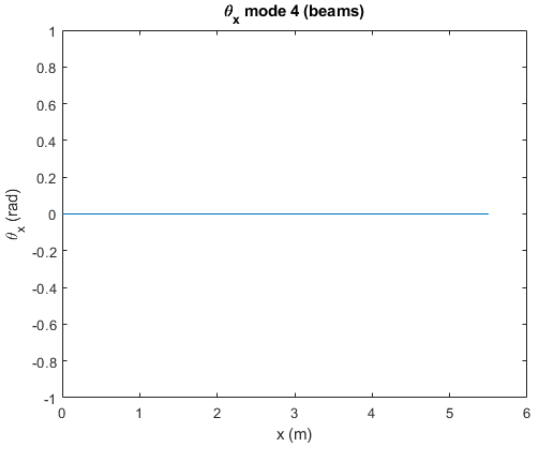


Figure 24: Fourth mode twist angle shape along x axis for beam discretization

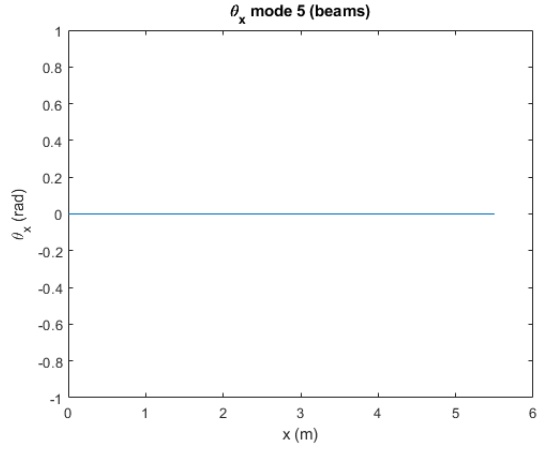


Figure 25: Fifth mode twist angle shape along x axis for beam discretization

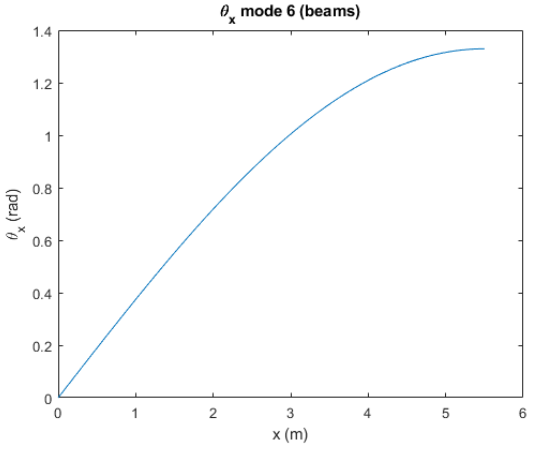


Figure 26: Sixth mode twist angle shape along x axis for beam discretization

2.1.2 Shell discretization of the wingbox

1. Vertical deflection

   As in the beam discretization, the vertical deflection for the bending test is presented first. Note the result is the same for both discretizations (compare figures 5 and 28).

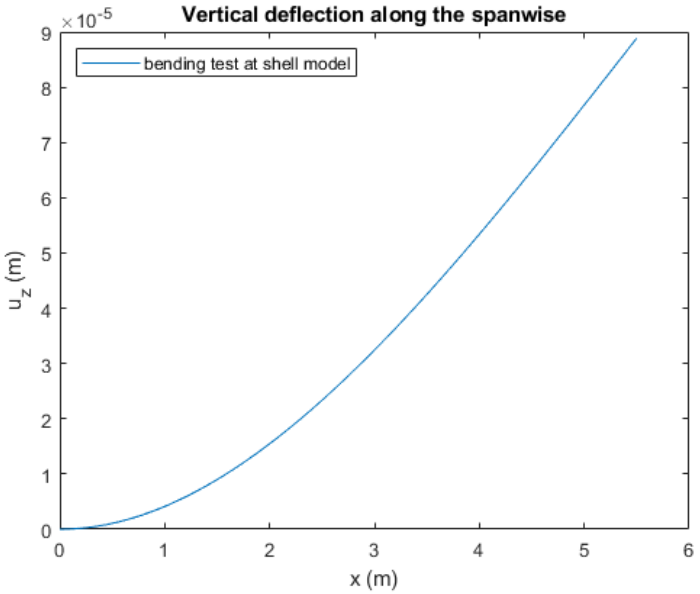


Figure 27: Vertical deflection distribution along the spanwise for the bending force and shell discretization

The average vertical deflection for the torsion case and shell discretization is zero, like in the other discretization and as expected.

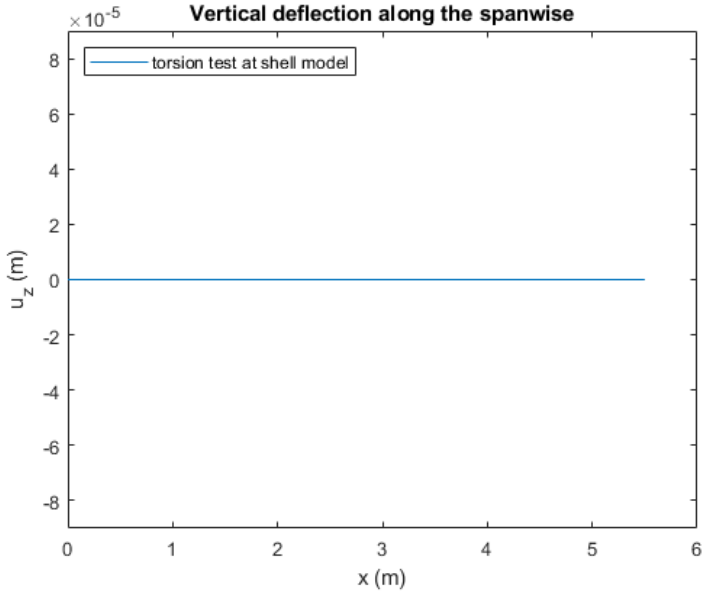


Figure 28: Vertical deflection distribution along the spanwise for the torsion force and shell discretization

2. Twist angle

   When checking the twist angle along the wingbox for the bending test it seems the deformed shape is not correct because, as explained before, during this test no twist is expected. The resulting plot is the following:

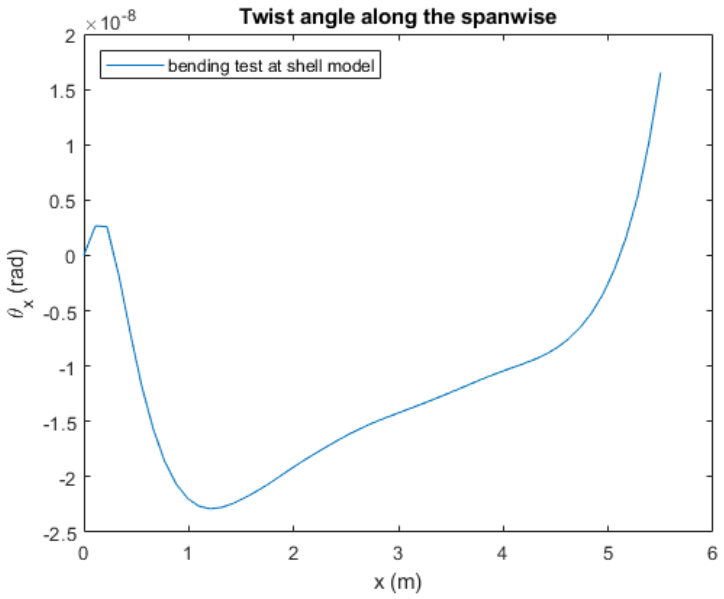


Figure 29: Twist angle distribution along the spanwise for the bending force and shell discretization

   However, after a detailed analysis of the result, one finds out the order of magnitude of the twist, which is about $2 \cdot 10^{-8}$ rad. If one scales the axis so that it is representing the same magnitudes as in the previous torsion cases (see figure 8) then the result is perceived as zero, as expected.

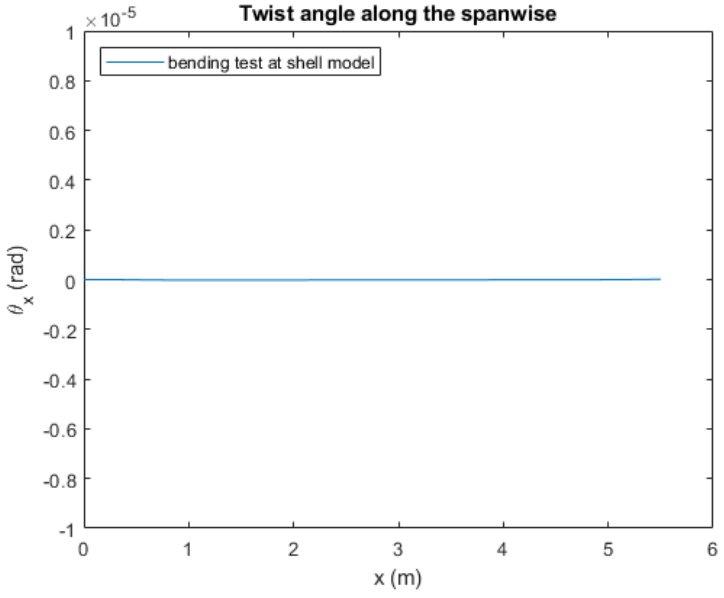


Figure 30: Twist angle distribution along the spanwise for the bending force and shell discretization with a more contracted vertical axis

The average twist distribution for the torsion case and shell discretization is probably the most interesting result. From the fixed tip to 3.5 meters the wingbox θ_x deformed shape is the same as in the beam discretization (see figures 8 and 31). However, the linear behaviour ends from a span of about 3.5 meters and is converted into an exponential zone. In other words, it's like the wingbox has an easiness for being twisted as getting closer to the free tip.

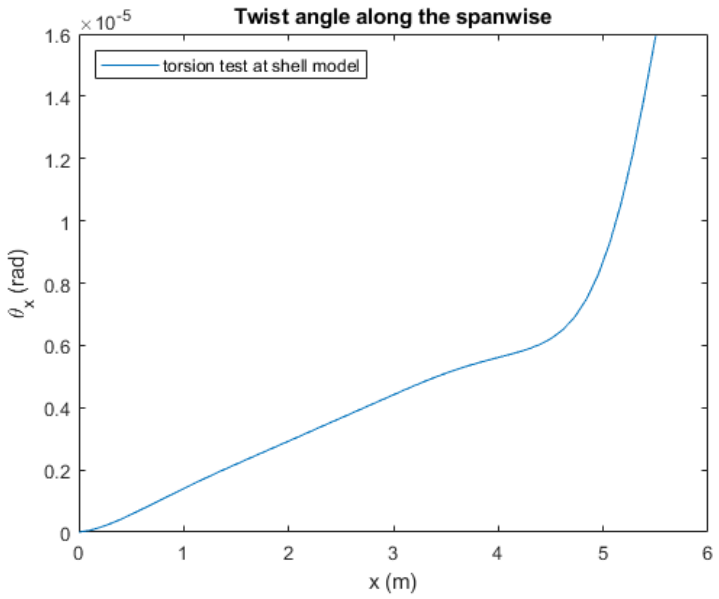


Figure 31: Twist angle distribution along the spanwise for the torsion force and shell discretization

It's important to note the result from the shell discretization is considered more accurate than the one from the beam due to the better discretization of the wingbox as shells are 2D elements whereas beams are only 1D, hence shell method is more reliable.

3. Modal analysis

   The same properties as before have been computed, allowing a comparison between the two discretizations:

f_1 (Hz)	f_2 (Hz)	f_3 (Hz)	f_4 (Hz)	f_5 (Hz)	f_6 (Hz)
3.86	11.64	23.99	66.15	70.62	79.53

Table 3: Frequencies of the first six natural modes of the wingbox modelled with shells

The first five natural frequencies are almost identical for both methodologies as they differ only in the second decimal. The sixth one is roughly 4 Hz lower in the shells case, considered more reliable due to the higher complexity on the discretization.

u_y modal analysis for shell discretization

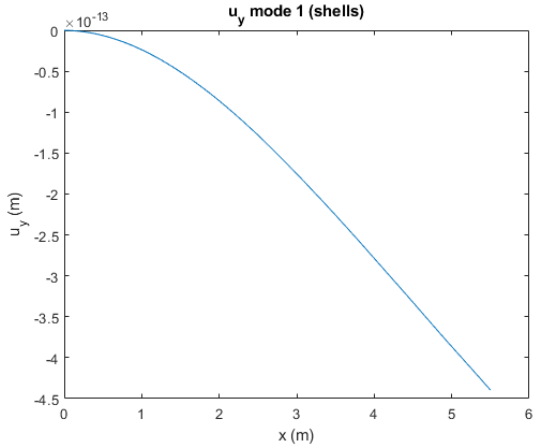


Figure 32: First mode deformed shape along y axis for shell discretization

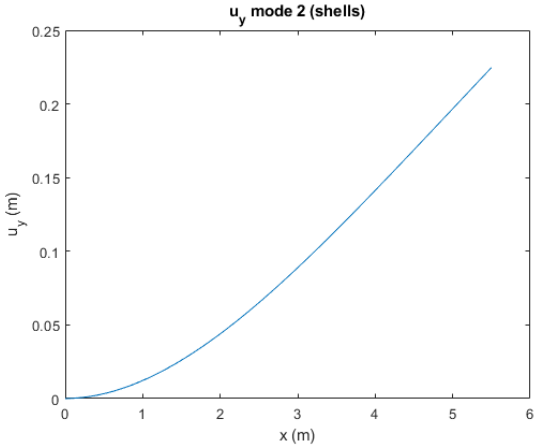


Figure 33: Second mode deformed shape along y axis for shell discretization

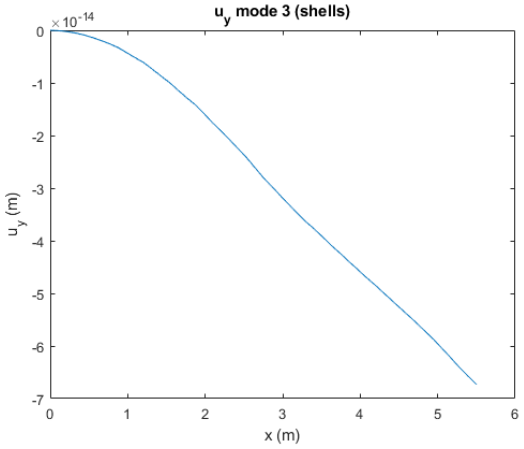


Figure 34: Third mode deformed shape along y axis for shell discretization

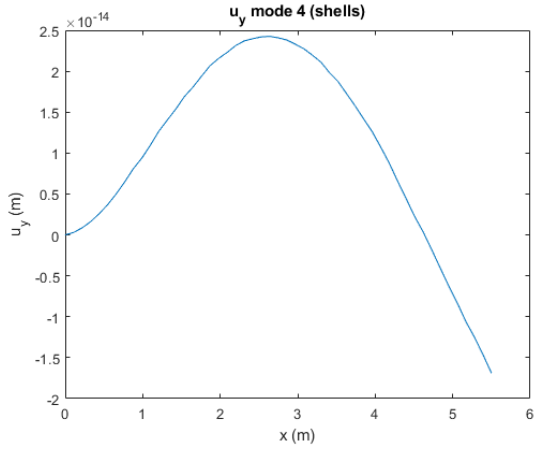


Figure 35: Fourth mode deformed shape along y axis for shell discretization

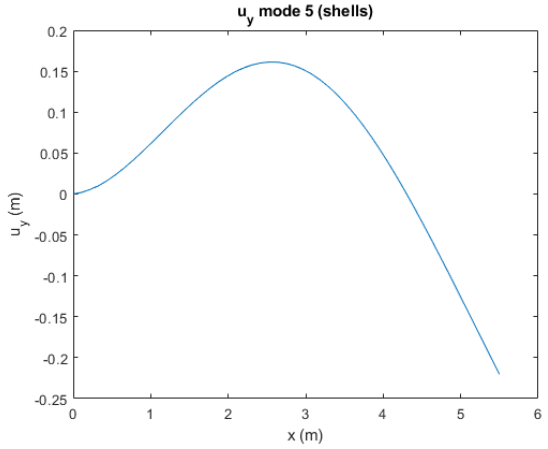


Figure 36: Fifth mode deformed shape along y axis for shell discretization

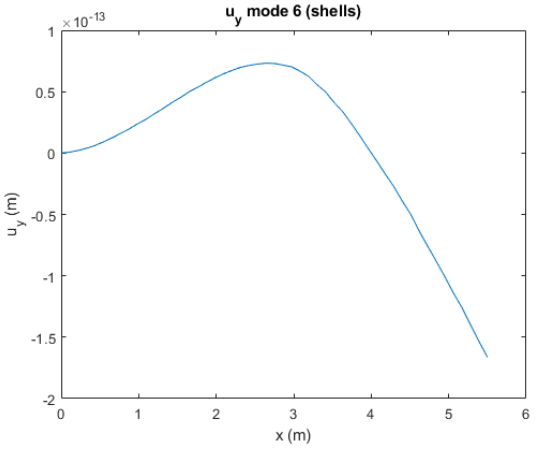


Figure 37: Sixth mode deformed shape along y axis for shell discretization

u_z modal analysis for shell discretization

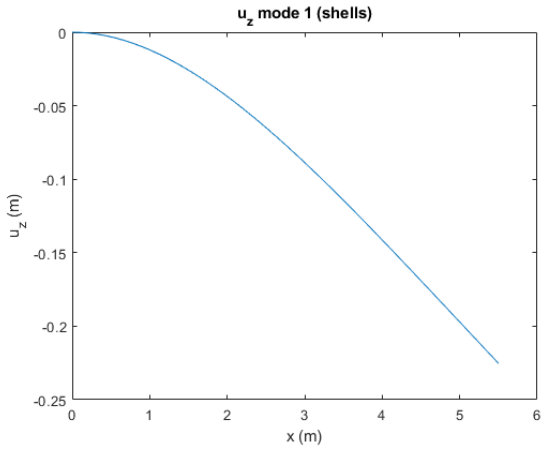


Figure 38: First mode deformed shape along z axis for shell discretization

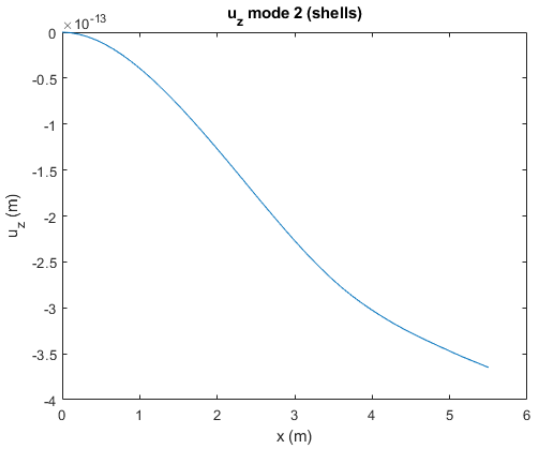


Figure 39: Second mode deformed shape along z axis for shell discretization

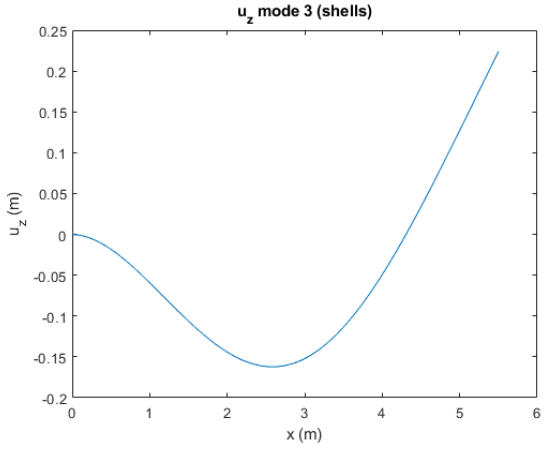


Figure 40: Third mode deformed shape along z axis for shell discretization

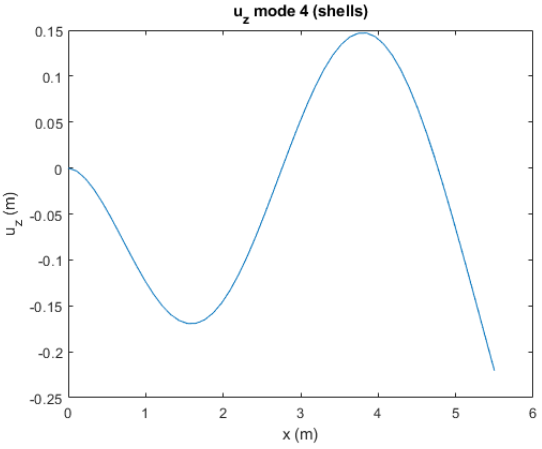


Figure 41: Fourth mode deformed shape along z axis for shell discretization

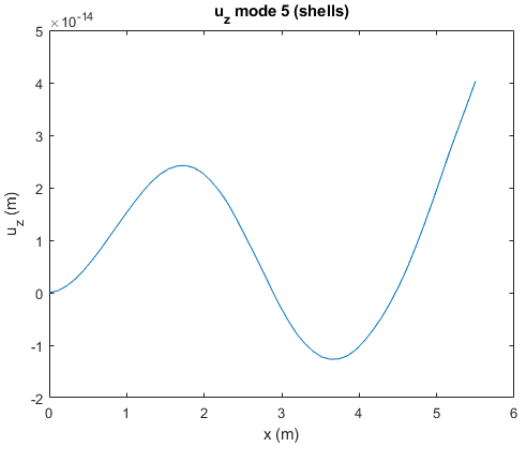


Figure 42: Fifth mode deformed shape along z axis for shell discretization

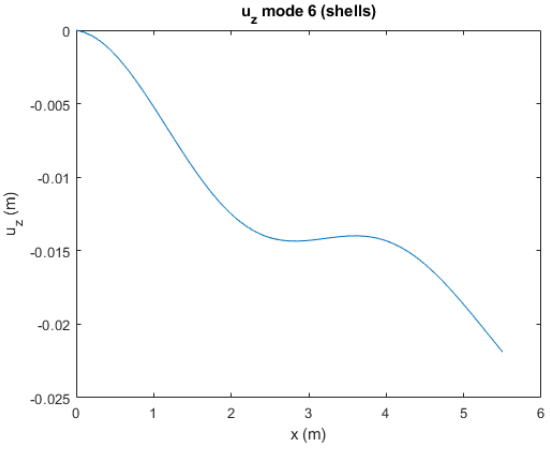


Figure 43: Sixth mode deformed shape along z axis for shell discretization

θ_x modal analysis for shell discretization

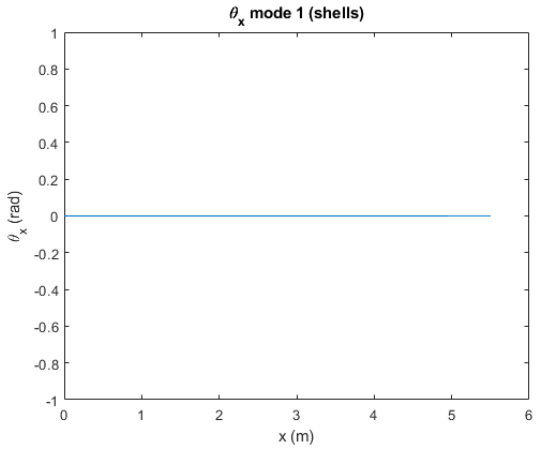


Figure 44: First mode twist angle shape along x axis for shell discretization

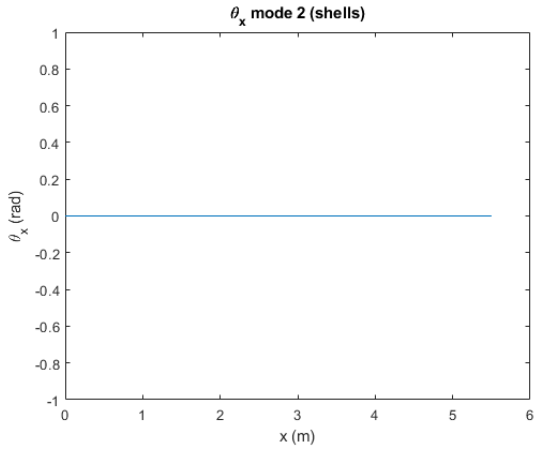


Figure 45: Second mode twist angle shape along x axis for shell discretization

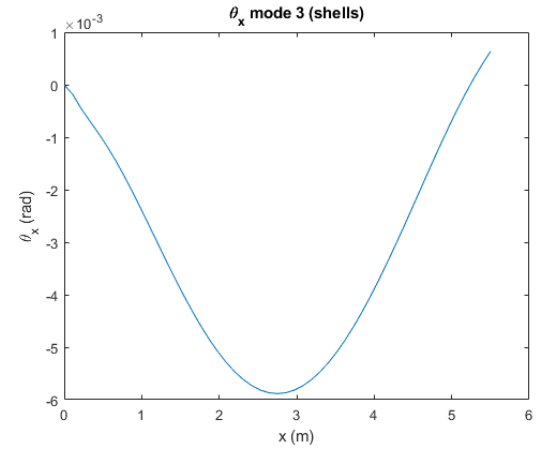


Figure 46: Third mode twist angle shape along x axis for shell discretization

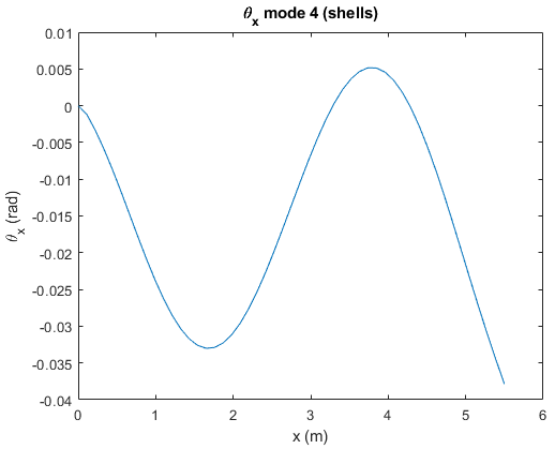


Figure 47: Fourth mode twist angle shape along x axis for shell discretization

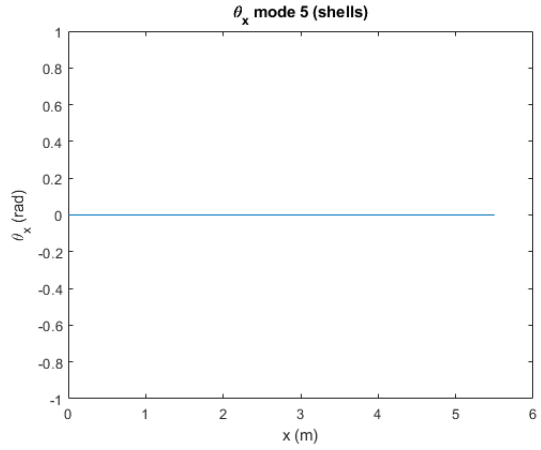


Figure 48: Fifth mode twist angle shape along x axis for shell discretization

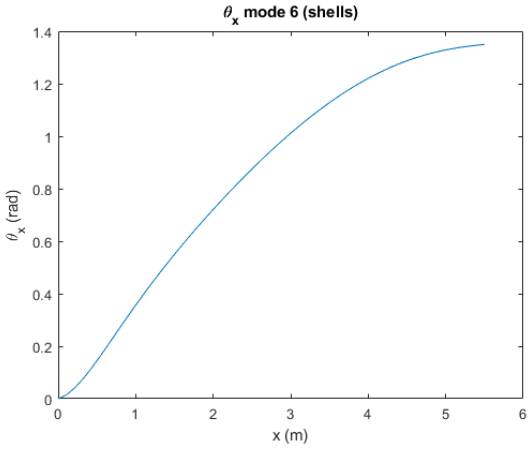


Figure 49: Sixth mode twist angle shape along x axis for shell discretization

2.2 Part II results: Whole wing structure

2.2.1 Bending and torsion tests

The aim of this section is to obtain the same information as in *Part I* but with a more realistic wing model. For instance, there is a wingbox as well as some ribs, stringers and skin as stated in the introduction (see figure 3 for more detail). The analyses have been performed considering all or some of the components combined, as explained further below.

First, the bending test has been considered. In the following page the results are presented, where the first column accounts for the vertical deflection and the second for the twist angle, making easier the comparison of the results because the first row only considers the wingbox, the second considers the wingbox and the ribs, and finally the third considers all the wing elements (wingbox, ribs, stringers and skin).

It can be seen how the bending test considering only the wingbox yields the same results as in *part I*, as expected.

The shape, regarding the vertical deflection u_z, of the wing is always the same. However, the maximum value decreases when more elements are considered. This behaviour was expected because the additional elements give more stiffness to the structure.

On the other hand, the twist is always zero as in *Part I*. Again, this is because the bending force is applied in the z direction in a perfectly balanced way.

It's important to note that the abbreviations of the plot legends are the same as those used in the assignment statement: WB for wingbox, RB for ribs, ST for stringers and SK for skin.

For the torsion case it seems the vertical deflection is very different from one case to another. However by checking the axis values one can conclude the deformed shape is so small that can be considered zero. Hence, the result is in line with *Part I*.

The twist of the torsion test is more interesting. The result for the wingbox alone is the same as in *Part I*, verifying the code performs well. By adding the ribs, the first half of the wing does not suffer torsion at all and the value at the tip is smaller. The result considering the complete wing (wingbox, ribs, stringers and skin) is practically the same as in the previous case: the deformed shape is the same while the difference of the twist at the tip is minimum.

Wing results plots for the bending test

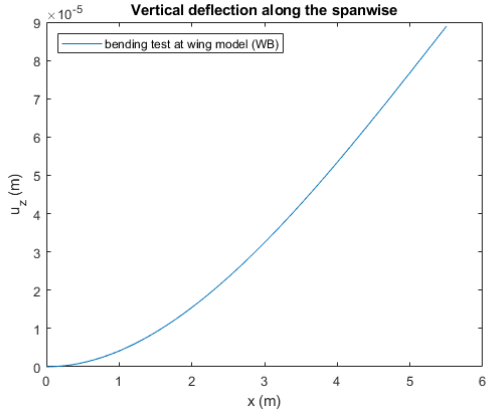


Figure 50: Average vertical deflection for the bending test considering the wingbox

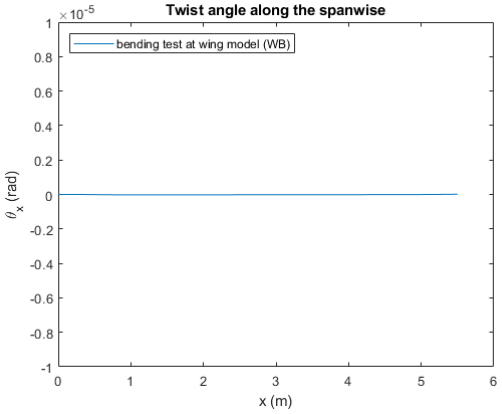


Figure 51: Average twist for the bending test considering the wingbox

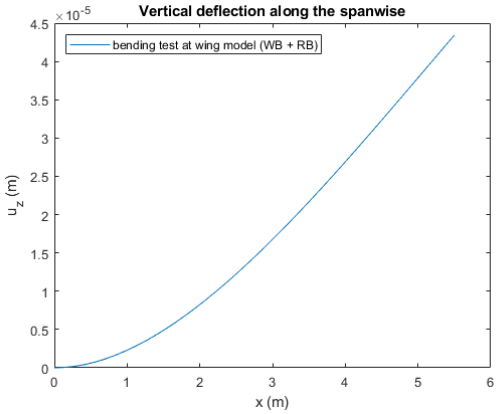


Figure 52: Average vertical deflection for the bending test considering the wingbox and the ribs

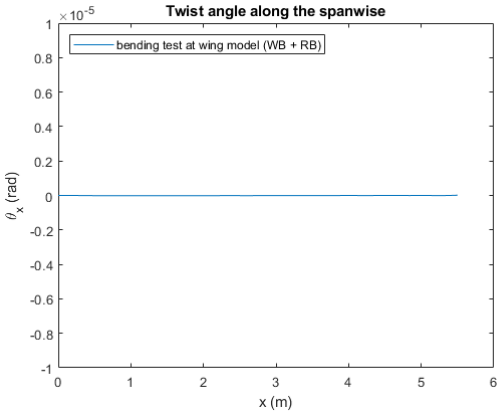


Figure 53: Average twist for the bending test considering the wingbox and the ribs

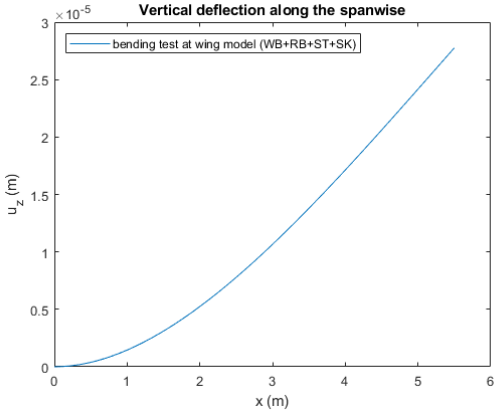


Figure 54: Average vertical deflection for the bending test considering the wingbox, ribs, stringers and skin

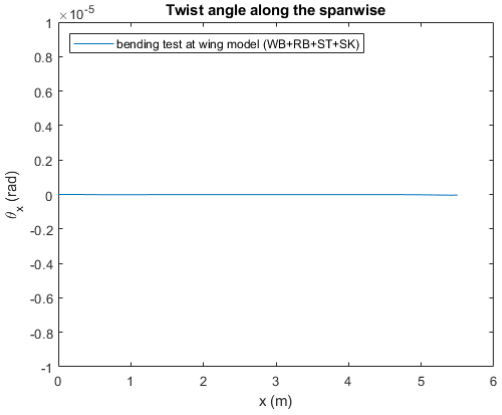


Figure 55: Average twist for the bending test considering the wingbox, ribs, stringers and skin

Wing results plots for the torsion test

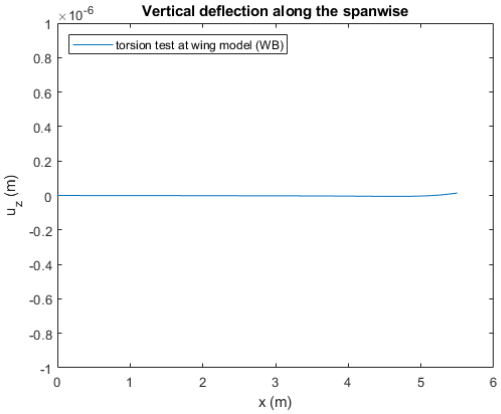


Figure 56: Average vertical deflection for the torsion test considering the wingbox

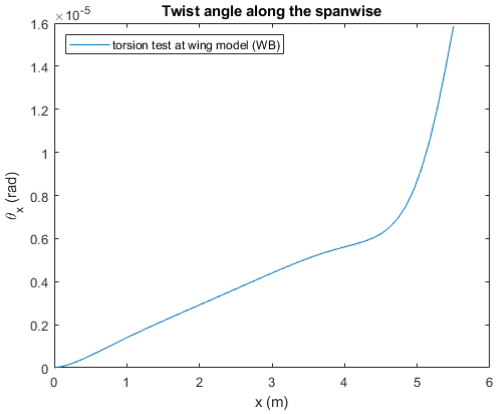


Figure 57: Average twist for the torsion test considering the wingbox

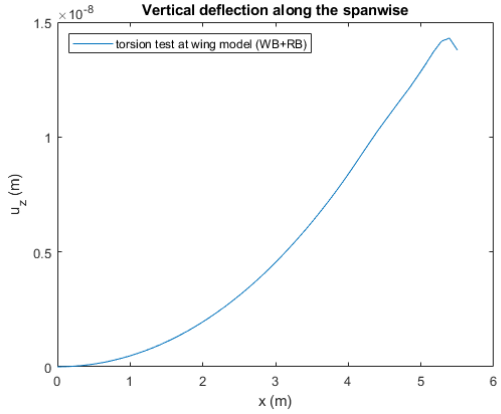


Figure 58: Average vertical deflection for the torsion test considering the wingbox and the ribs

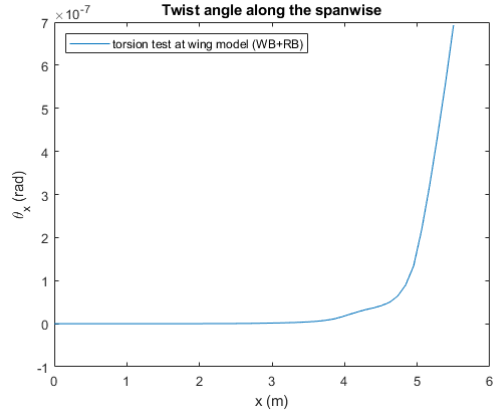


Figure 59: Average twist for the torsion test considering the wingbox and the ribs

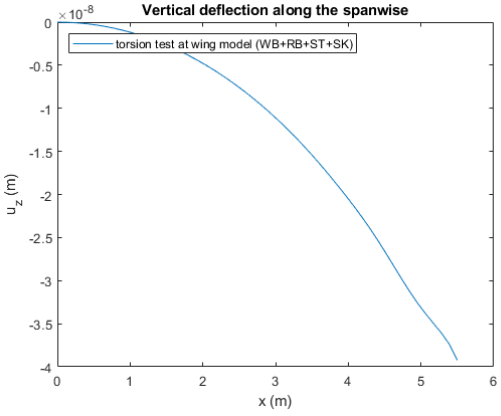


Figure 60: Average vertical deflection for the torsion test considering the wingbox, ribs, stringers and skin

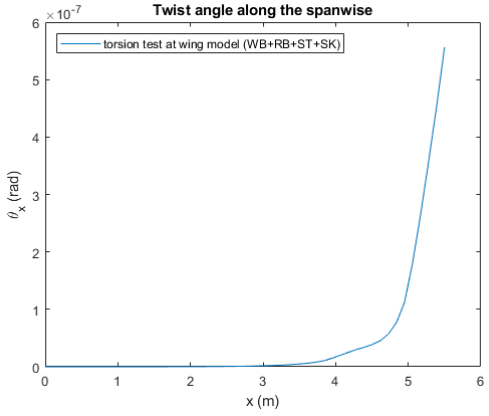


Figure 61: Average twist for the torsion test considering the wingbox, ribs, stringers and skin

2.2.2 Wind tunnel test using direct solving of the equations

1. Solution of the static problem through the direct solving of the system of equations

   The wing can be observed to exhibit mainly a bending behavior upon the wind tunnel loads analysis. The bending is upwards, meaning the aerodynamic loads generate a positive lift (to be expected given that the airfoil is symmetric and that the angle of attack is 10°) and that the lift is larger than the weight of the structure.

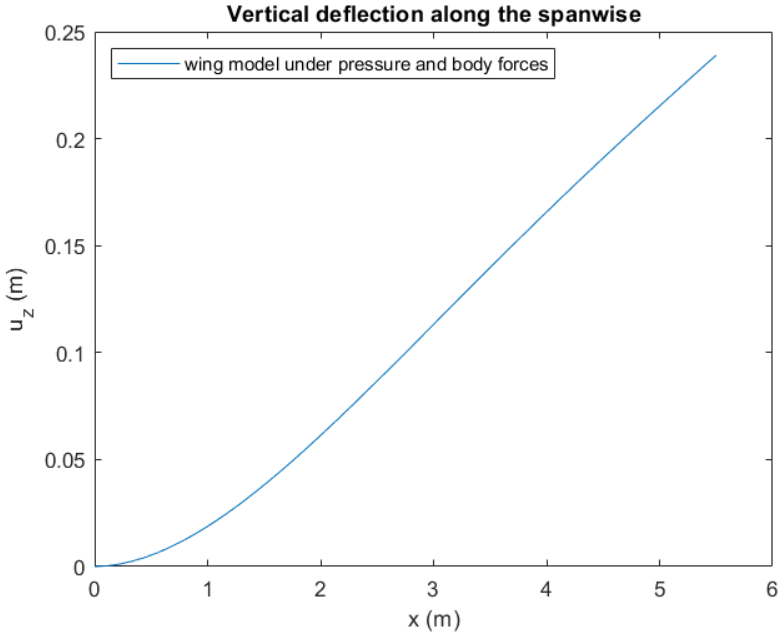


Figure 62: Vertical deflection distribution along the spanwise for the wind tunnel test on the wing

The twist angle is observed to be very small, taking values of the order of $10^{-4}\,rad$.

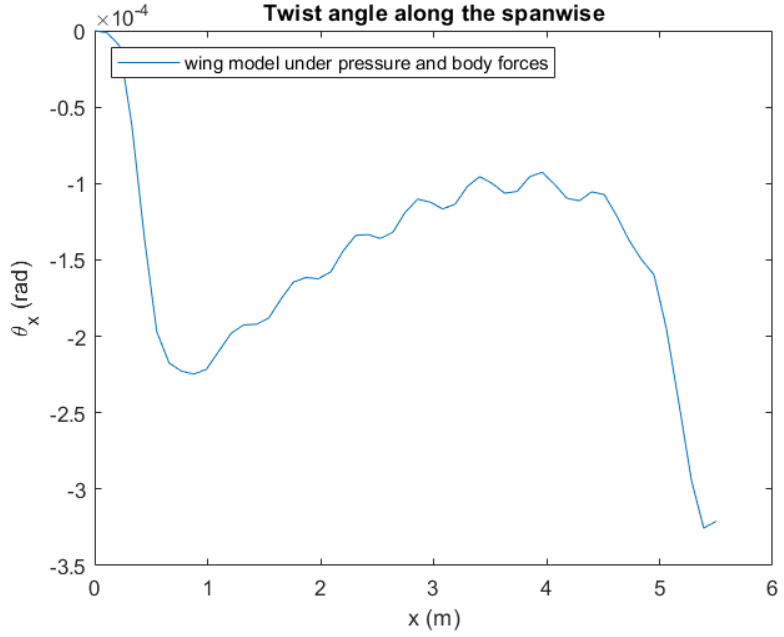


Figure 63: Twist angle distribution along the spanwise for the wind tunnel test on the wing

2. Von Mises stress distribution in the wingbox, ribs and skin elements

   The Von Mises stress distribution in the wingbox, ribs and skin elements has been extracted and plotted together with the deformed shape of the wing.

   As expected, it can be seen how the stress on the skin and wingbox increases with the proximity to the root of the wing. The highest stress on the entire wing structure can be observed to occur at the ribs, where there exist several stress concentration regions that reach a maximum of 1003 MPa and occur at the interface with the wingbox/stringers (see close-up figure). This is a value that is exceedingly high for most materials. However, as these are stress concentrations at very specific locations near the interfaces, it would be important to analyze the interfaces in more detail to be sure that those stresses are real and not due to simplifications in the model. On another note, the stresses on the wingbox and skin however do not exceed 162 MPa, which can be handled by most metals.

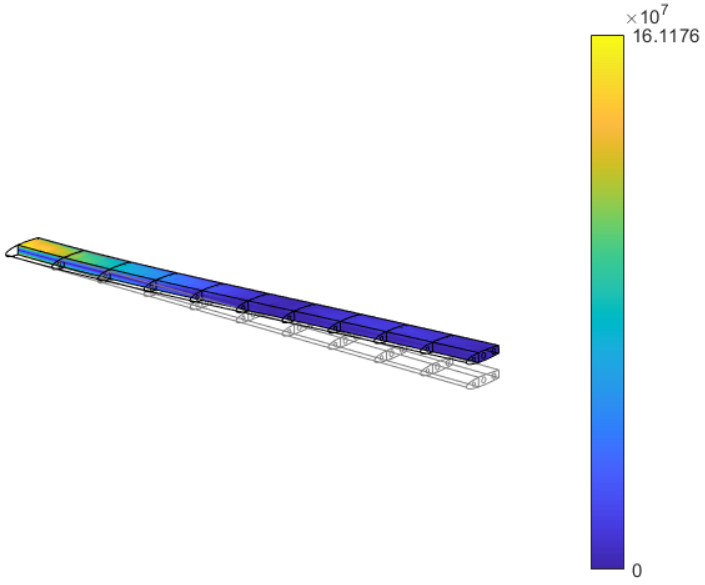


Figure 64: Von Mises stress on the wing box over the deformed shape for the wind tunnel test. Scale: 1, units: Pa

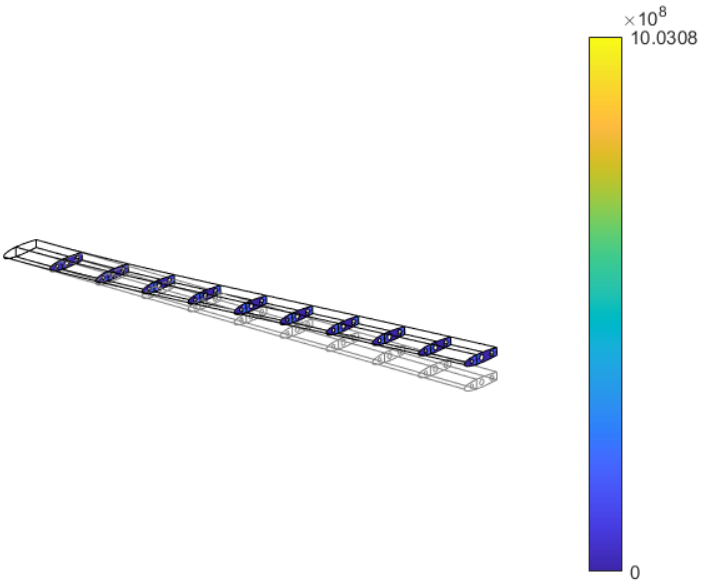


Figure 65: Von Mises stress on the ribs over the deformed shape for the wind tunnel test. Scale: 1, units: Pa

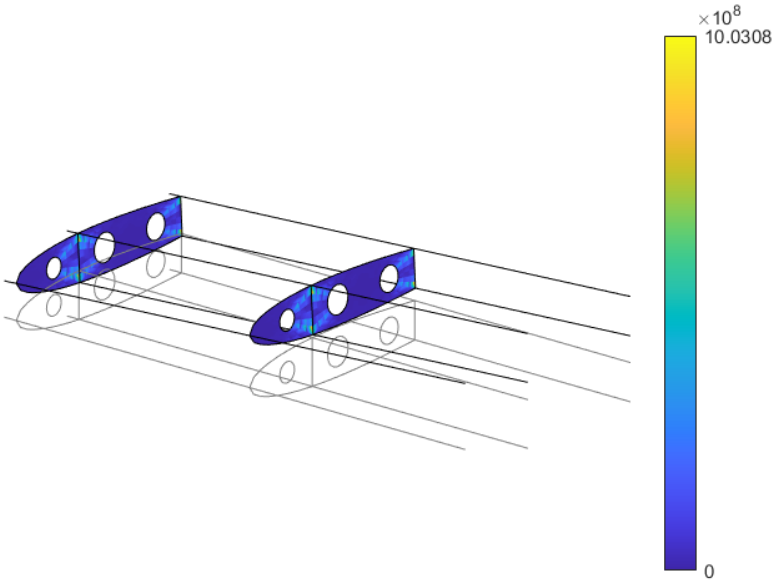


Figure 66: Von Mises stress on the ribs over the deformed shape for the wind tunnel test. Close-up figure. Scale: 1, units: Pa

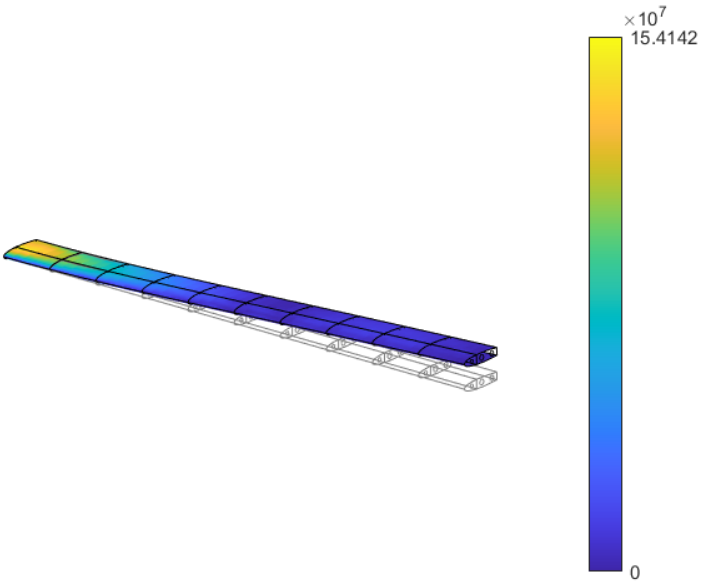


Figure 67: Von Mises stress on the skin over the deformed shape for the wind tunnel test. Scale: 1, units: Pa

2.2.3 Wind tunnel test using a model-order reduction strategy

1. Modal analysis

   The first step to selecting the appropriate modes for the modal projection is to perform a modal analysis. The following figure shows the first 9 modes of the model.

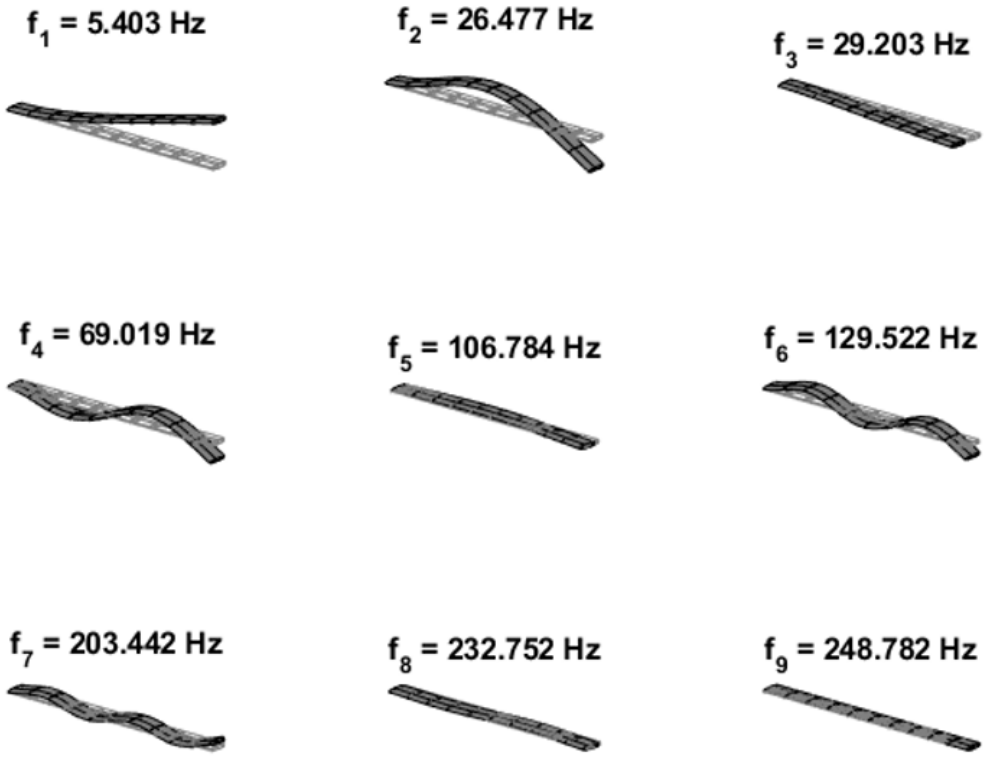


Figure 68: Modes 1 to 9 of the wing structure.

Based on the previous results with the direct solving it was seen that the wind tunnel loads mainly cause a bending of the wing structure upwards and that the twist of the wing is negligible as well as the lateral bending. This would mean that of the first 9 modes, only modes 1, 2, 4, 6 and 7 would apply. However, upon some testing, analyses using also mode 3 (first lateral bending mode) have shown greatly improved results, meaning mode 3 is also relevant. This may mean that while not appreciable in the previous plots, the static response of the wing under the wind tunnel test exhibits some lateral bending.

Below are shown the results of average vertical deflection and twist along the wingspan as well as of the Von Mises stress on the different components of the wing. It can be appreciated how both the deformations and stresses take values very close to those obtained with the direct solution.

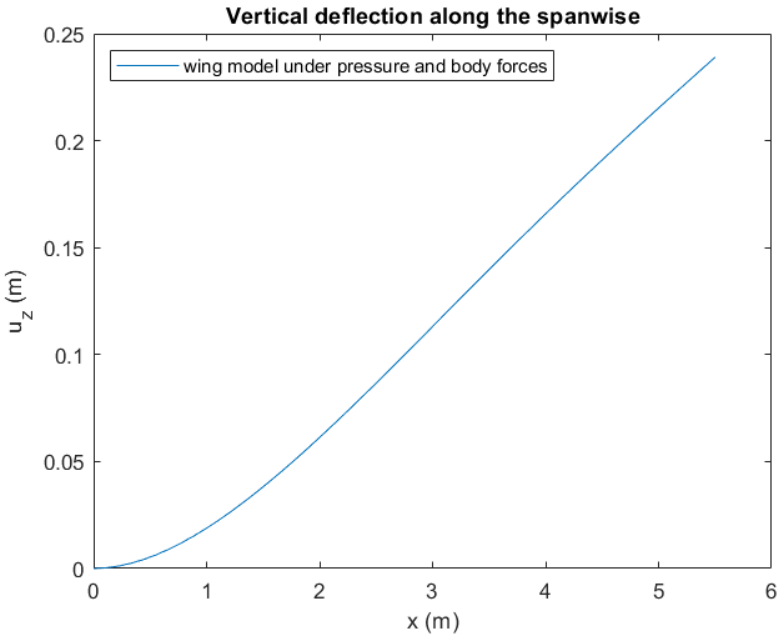


Figure 69: Vertical deflection distribution along the spanwise for the wind tunnel test on the wing. Obtained with modal projection

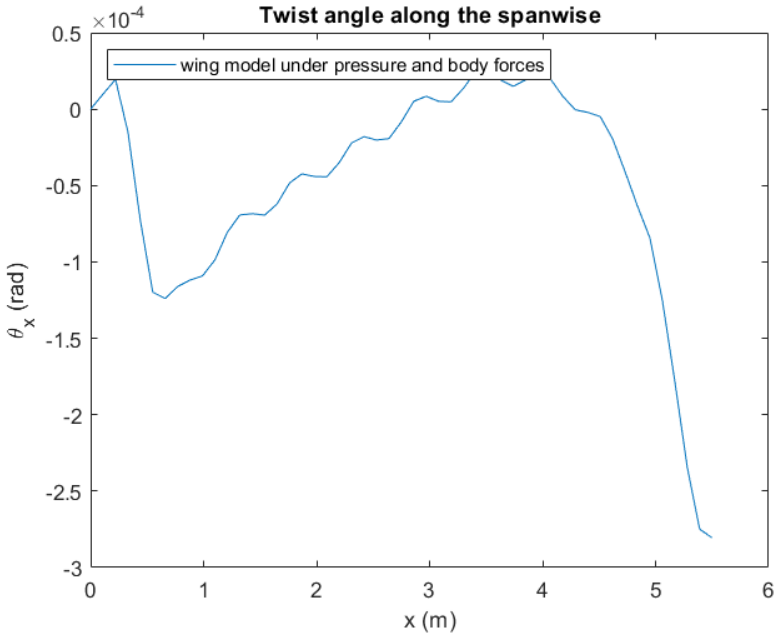


Figure 70: Twist angle distribution along the spanwise for the wind tunnel test on the wing. Obtained with modal projection

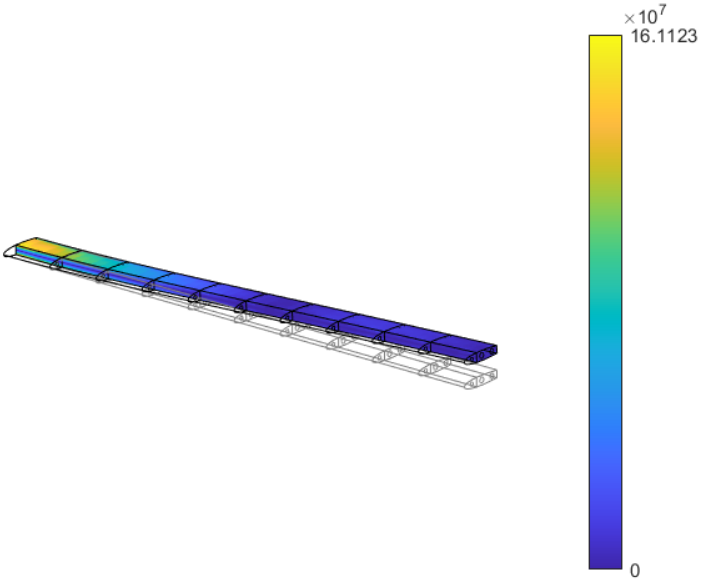


Figure 71: Von Mises stress on the wing box over the deformed shape for the wind tunnel test. Scale: 1, units: Pa. Obtained with modal projection

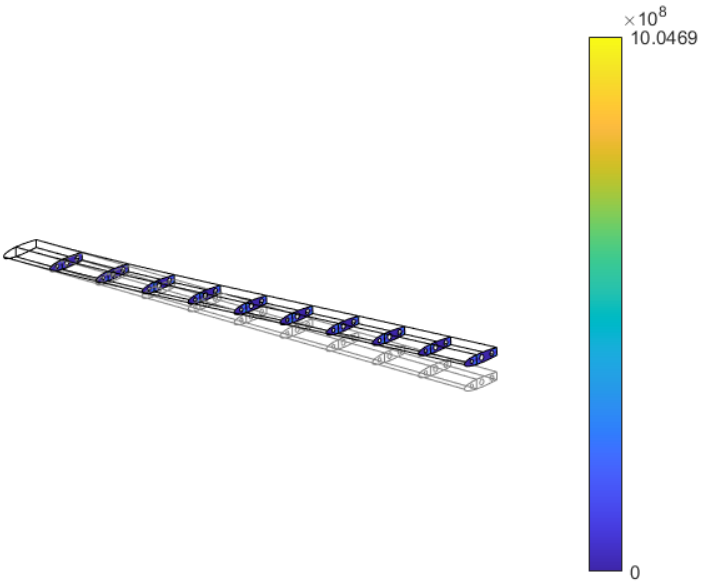


Figure 72: Von Mises stress on the ribs over the deformed shape for the wind tunnel test. Scale: 1, units: Pa. Obtained with modal projection

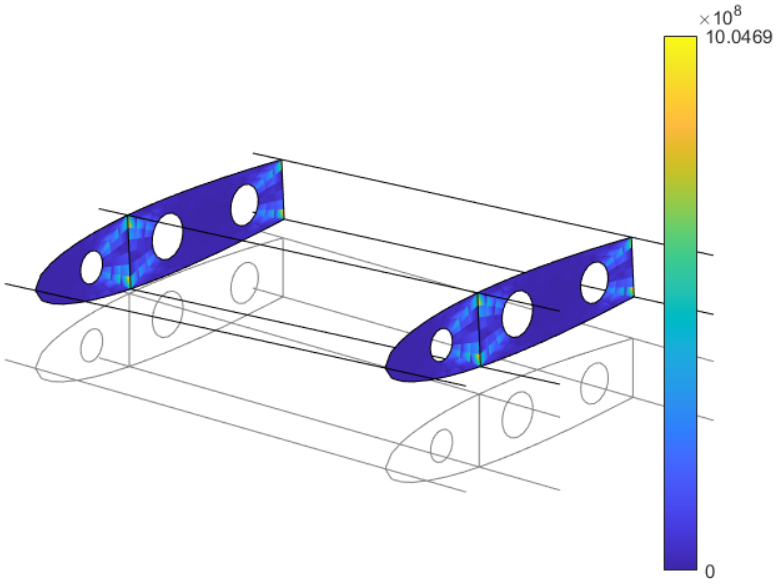


Figure 73: Von Mises stress on the ribs over the deformed shape for the wind tunnel test. Close-up figure. Scale: 1, units: Pa. Obtained with modal projection

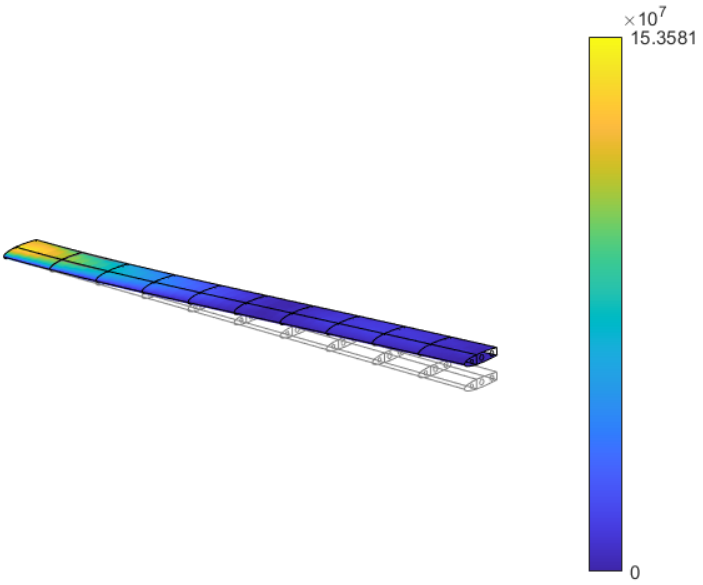


Figure 74: Von Mises stress on the skin over the deformed shape for the wind tunnel test. Scale: 1, units: Pa. Obtained with modal projection

3 Conclusions

This project has culminated with the resolution of a representative wing structure formed by 4 different components and which use two different discretization types (shells for the wingbox, ribs and skin, beam modeling for the stringers) under a real loading case which is a combination of the pressure loads generated by the aerodynamic forces and the weight as a body force.

To reach this point, this project began with a simplified problem that considered only the wingbox modeled with beam elements. This model has been subjected to two different load cases with simple unitary point loads. One is a vertical force and the other is a torque. Upon the verification that results coherent with the problem physics had been obtained, a more complex case was analyzed which was the wingbox modeled with shells. This second case was subjected to the same loads as before, but, to apply them, they were converted into a system of two equivalent forces applied at two nodes of the shell model instead of one node. The results with this new discretization were verified to be coherent with those obtained for the beam model, giving confidence in the results.

After these analyses, the entire wing model was prepared. It was first subjected to the same loading conditions as the case with the isolated wingbox, and considering only some components and then all, to see the effects of the different components. For example, it was observed how the addition of the ribs had a great impact on reducing the twist when the wing was subjected to torque.

Finally, the pressure and weight load case was analyzed on the entire wing. It was seen how, as expected, it mainly caused a bending of the wing. It was also seen how the pressure loads are positive in the vertical direction and greater than the weight of the wing, as the wing deflected upwards. For the pressure forces to be positive was to be expected given that the case is of a symmetric airfoil with a positive angle of attack, which generates positive lift.

The Von Mises stresses on the wingbox and skin were observed to increase with the proximity to the root of the wing and the maximum stress on them is observed to take an acceptable value (less than 162 MPa). However, exceedingly high stresses were obtained on the ribs, with a value of 1003 MPa

(too high for most materials). That said, these stresses occur in specific areas of stress concentration near the interfaces with the wingbox/stringers, and thus, the value obtained may not be the true value of stress and may be simply due to the simplifications on the model and the mesh density used. It is proposed, as future work, to create a more detailed model with a more detailed mesh, with special attention to the modeling of the interfaces, to solve the uncertainty and assess whether these stresses are representative of the real case.

Finally, the wind tunnel case was solved again but using a model-order reduction strategy based on a modal projection, following a modal analysis where the most relevant modes were selected. Very similar results to the direct solution were obtained, giving confidence in the method and modes selected.

4 Annex

4.1 MAIN BEAM

```
%% PART I - BEAM MODELLING (TEMPLATE)

% Useful commands to initialize script
clear
close all
clear all

%% DATA

problem_type = 1; %1 for bending. 2 for torsion.
Nm = 6; %number of modes

% Define the problem's data (e.g. dimensions, material properties, etc.)
E = 71e9; %Pa
v_poisson = 0.33;
G = E/(2*(1+v_poisson));
rho = 2300; %kg/m3

c = 0.8; %m
b = 5.5; %m
y0 = 0.3083; %m
y1 = 0.176; %m
y2 = 0.464; %m
h1 = 17.5; %m
h2 = 15; %m
h3 = 6; %m
A = 0.0062; %m2
Iy = 8.801e-6; %m4
Iz = 8.069e-5; %m4
J = 8.949e-5; %m4
ky = 0.5457;
kz = 0.1767;
kt = 0.2920;
yc = 0.2954; %m

%% PREPROCESS

% Load mesh data
load('beam.mat','xn','Tn','Tm');
% xn : Nodal coordinates
%       |x1 y1 z1|
%       |x2 y2 z2|
%       |  ...   |
% Tn : Nodal connectivities [Nelem x 2]
% Tm : Material connectivities [Nelem x 1]

% Define boundary conditions and forces data matrices: Up, Fe, Qe, Be
N_elements = size(Tn,1);
N_nodes = size(xn,1);
Up = [0 1 1;
      0 1 2;
      0 1 3;
      0 1 4;
      0 1 5;
      0 1 6];

switch(problem_type)
    case 1
```

```
    Fe = [1 N_nodes 3];
    case 2
    Fe = [1 N_nodes 4];
end

Qe = [0 1 2];

Be = [0 1 2];

j_orientations = [0 1 0];

E_vector = [E]; %more rows would be added if more values existed
A_vector = [A]; %more rows would be added if more values existed
G_vector = [G]; %more rows would be added if more values existed
Iy_vector = [Iy]; %more rows would be added if more values existed
Iz_vector = [Iz]; %more rows would be added if more values existed
ky_vector = [ky]; %more rows would be added if more values existed
kz_vector = [kz]; %more rows would be added if more values existed
kt_vector = [kt]; %more rows would be added if more values existed
J_vector = [J]; %more rows would be added if more values existed
rho_vector = [rho]; %more rows would be added if more values existed


%% SOLVER

% Obtain system matrices

% TIP: To avoid recomputing the system matrices use a save/load structure:
if 1 % Set to 1 to (re)compute the system matrices and '0' to load them

    % Compute system matrices (as long as parameters don't change there is
    % no need to repeat the matrix assembly on every run)

    %--section 2----------------------------------------
    Ndof = 6*N_nodes;
    K=zeros(Ndof,Ndof);
    M=zeros(Ndof,Ndof);

    for e=1:1:N_elements

        l(e) = norm(xn(Tn(e,2),:)-xn(Tn(e,1),:));
        i = ((xn(Tn(e,2),:))' - (xn(Tn(e,1),:))')/l(e);
        j = j_orientations(Tm(e,1),:)';
        k = cross(i,j);
        Rp = [i(1,1) j(1,1) k(1,1) 0 0 0;
              i(2,1) j(2,1) k(2,1) 0 0 0;
              i(3,1) j(3,1) k(3,1) 0 0 0;
              0 0 0 i(1,1) j(1,1) k(1,1);
              0 0 0 i(2,1) j(2,1) k(2,1);
              0 0 0 i(3,1) j(3,1) k(3,1)]';

        R(:,:,e) = [Rp(1,1) Rp(1,2) Rp(1,3) Rp(1,4) Rp(1,5) Rp(1,6) 0 0 0 0 0 0;
                    Rp(2,1) Rp(2,2) Rp(2,3) Rp(2,4) Rp(2,5) Rp(2,6) 0 0 0 0 0 0;
                    Rp(3,1) Rp(3,2) Rp(3,3) Rp(3,4) Rp(3,5) Rp(3,6) 0 0 0 0 0 0;
                    Rp(4,1) Rp(4,2) Rp(4,3) Rp(4,4) Rp(4,5) Rp(4,6) 0 0 0 0 0 0;
                    Rp(5,1) Rp(5,2) Rp(5,3) Rp(5,4) Rp(5,5) Rp(5,6) 0 0 0 0 0 0;
                    Rp(6,1) Rp(6,2) Rp(6,3) Rp(6,4) Rp(6,5) Rp(6,6) 0 0 0 0 0 0;
                    0 0 0 0 0 0 Rp(1,1) Rp(1,2) Rp(1,3) Rp(1,4) Rp(1,5) Rp(1,6);
                    0 0 0 0 0 0 Rp(2,1) Rp(2,2) Rp(2,3) Rp(2,4) Rp(2,5) Rp(2,6);
                    0 0 0 0 0 0 Rp(3,1) Rp(3,2) Rp(3,3) Rp(3,4) Rp(3,5) Rp(3,6);
                    0 0 0 0 0 0 Rp(4,1) Rp(4,2) Rp(4,3) Rp(4,4) Rp(4,5) Rp(4,6);
                    0 0 0 0 0 0 Rp(5,1) Rp(1,2) Rp(5,3) Rp(5,4) Rp(5,5) Rp(5,6);
                    0 0 0 0 0 0 Rp(6,1) Rp(6,2) Rp(6,3) Rp(6,4) Rp(6,5) Rp(6,6)];
```

```
123
124         Nx(1)=-1/l(e);
125         Nx(2)=1/l(e);
126         Ba(1,:,e) = [Nx(1) 0 0 0 0 0 Nx(2) 0 0 0 0 0];
127         Ca = E_vector(Tm(e,1),1)*A_vector(Tm(e,1),1);
128         Ka(:,:,e) = l(e)*R(:,:,e)'*Ba(1,:,e)'*Ca*Ba(1,:,e)*R(:,:,e);
129
130         Bb(:,:,e) = [0 0 0 0 Nx(1) 0 0 0 0 0 Nx(2) 0;
131                      0 0 0 0 0 Nx(1) 0 0 0 0 0 Nx(2)];
132
133         Cb = E_vector(Tm(e,1),1)*[Iy_vector(Tm(e,1),1) 0;
134                                   0 Iz_vector(Tm(e,1),1)];
135
136         Kb(:,:,e) = l(e)*R(:,:,e)'*Bb(:,:,e)'*Cb*Bb(:,:,e)*R(:,:,e);
137
138         N_Gauss = 1/2;
139
140         Bs(:,:,e) = [0 Nx(1) 0 0 0 -N_Gauss 0 Nx(2) 0 0 0 -N_Gauss;
141                      0 0 Nx(1) 0 N_Gauss 0 0 0 Nx(2) 0 N_Gauss 0];
142
143         Cs = G_vector(Tm(e,1),1)*A_vector(Tm(e,1),1)*[ky_vector(Tm(e,1),1) 0;
144                                                       0 kz_vector(Tm(e,1),1)];
145
146         Ks(:,:,e)=l(e)*R(:,:,e)'*Bs(:,:,e)'*Cs*Bs(:,:,e)*R(:,:,e);
147
148         Bt(1,:,e) = [0 0 0 Nx(1) 0 0 0 0 0 Nx(2) 0 0];
149         Ct = G_vector(Tm(e,1),1)*J_vector(Tm(e,1),1)*kt_vector(Tm(e,1),1);
150         Kt(:,:,e) = l(e)*R(:,:,e)'*Bt(1,:,e)'*Ct*Bt(1,:,e)*R(:,:,e);
151
152         Xi = [-1/sqrt(3);
153                 1/sqrt(3)];
154         w = [1;
155              1];
156
157         rho_matrix = rho_vector(Tm(e,1),1)*[A_vector(Tm(e,1),1) 0 0 0 0 0;
158                                             0 A_vector(Tm(e,1),1) 0 0 0 0;
159                                             0 0 A_vector(Tm(e,1),1) 0 0 0;
160                                             0 0 0 J_vector(Tm(e,1),1) 0 0;
161                                             0 0 0 0 Iy_vector(Tm(e,1),1) 0;
162                                             0 0 0 0 0 Iz_vector(Tm(e,1),1)];
163         Me(:,:,e)=zeros(12,12);
164
165         for kk=1:1:2
166             N(1) = (1-Xi(kk))/2;
167             N(2) = (1+Xi(kk))/2;
168             N_matrix(:,:,e,kk)=[N(1) 0 0 0 0 0 N(2) 0 0 0 0 0;
169                                 0 N(1) 0 0 0 0 0 N(2) 0 0 0 0;
170                                 0 0 N(1) 0 0 0 0 0 N(2) 0 0 0;
171                                 0 0 0 N(1) 0 0 0 0 0 N(2) 0 0;
172                                 0 0 0 0 N(1) 0 0 0 0 0 N(2) 0;
173                                 0 0 0 0 0 N(1) 0 0 0 0 0 N(2)];
174             Me(:,:,e) = Me(:,:,e) + (w(kk)*l(e)*R(:,:,e)'*N_matrix(:,:,e,kk)'*
    rho_matrix*N_matrix(:,:,e,kk)*R(:,:,e))/2;
175
176         end
177
178         for jj=1:1:6
179             Idof(jj,1)=6*(Tn(e,1)-1)+jj;
180             Idof(6+jj,1)=6*(Tn(e,2)-1)+jj;
181         end
182
183         K(Idof,Idof)=K(Idof,Idof) + Ka(:,:,e) + Kb(:,:,e) + Ks(:,:,e) + Kt(:,:,e);
184         M(Idof,Idof)=M(Idof,Idof) + Me(:,:,e);
```

```
185
186     end
187
188
189         %--section 3---------------------------------------
190         f=zeros(Ndof,1);
191         for q=1:1:size(Fe,1)
192             f(6*(Fe(q,2)-1)+Fe(q,3),1) = f(6*(Fe(q,2)-1)+Fe(q,3),1) + Fe(q,1);
193         end
194
195         Q = zeros(N_nodes,6);
196
197         for r=1:1:size(Qe,1)
198             Q(Qe(r,2),Qe(r,3)) = Q(Qe(r,2),Qe(r,3)) + Qe(r,1);
199         end
200
201         B = zeros(N_nodes,6);
202
203         for s=1:1:size(Be,1)
204             B(Be(s,2),Be(s,3)) = B(Be(s,2),Be(s,3)) + Be(s,1);
205         end
206
207         for e=1:1:N_elements
208             bb(:,e) = [B(Tn(e,1),:),B(Tn(e,2),:)]';
209             qq(:,e) = [Q(Tn(e,1),:),Q(Tn(e,2),:)]';
210             fe(:,e) = Me(:,:,e)*bb(:,e);
211             for kk=1:1:2
212                 fe(:,e) = fe(:,e) + (w(kk)*l(e)*R(:,:,e)'*N_matrix(:,:,e,kk)'*
    N_matrix(:,:,e,kk)*R(:,:,e)*qq(:,e))/2;
213             end
214
215             for jj=1:1:6
216                 Idof(jj,1) = 6*(Tn(e,1)-1)+jj;
217                 Idof(6+jj,1) = 6*(Tn(e,2)-1)+jj;
218             end
219
220             f(Idof,1) = f(Idof,1) + fe(:,e);
221         end
222
223
224         %--section 4---------------------------------------
225         u = zeros(Ndof,1);
226         for p=1:1:size(Up,1)
227             Ip(p) = 6*(Up(p,2)-1) + Up(p,3);
228             u(Ip(p),1) = Up(p,1);
229         end
230         If = setdiff(1:Ndof,Ip);
231
232
233         %--section 5---------------------------------------
234         u(If,1) = K(If,If)\(f(If,1)-(K(If,Ip)*u(Ip,1)));
235         fR = K*u-f;
236
237
238         %--section 6---------------------------------------
239         for e=1:1:N_elements
240             for jj=1:1:6
241                 Idof(jj,1) = 6*(Tn(e,1)-1)+jj;
242                 Idof(6+jj,1) = 6*(Tn(e,2)-1)+jj;
243             end
244             ue = u(Idof,1);
245
246             epsilon_a(1,e) = Ba(1,:,e)*R(:,:,e)*ue;
```

```
            epsilon_s(:,e) = Bs(:,:,e)*R(:,:,e)*ue;
            epsilon_t(1,e) = Bt(1,:,e)*R(:,:,e)*ue;
            epsilon_b(:,e) = Bb(:,:,e)*R(:,:,e)*ue;

            fint(:,e) = R(:,:,e)*(Ka(:,:,e)+Kb(:,:,e)+Ks(:,:,e)+Kt(:,:,e))*ue;
            Fx(:,e) = [fint(1,e); -fint(7,e)];
            Fy(:,e) = [fint(2,e); -fint(8,e)];
            Fz(:,e) = [fint(3,e); -fint(9,e)];
            Mx(:,e) = [fint(4,e); -fint(10,e)];
            My(:,e) = [fint(5,e); -fint(11,e)];
            Mz(:,e) = [fint(6,e); -fint(12,e)];
        end


        %--section modal--------------------------------------

    [V,D] = MAIN_modal(K,M,If,Nm);

    Phi = zeros(Ndof,Nm);
    lambda = zeros(1,Nm);
    for kk=1:1:size(V,2)
        Phi(If,kk) = V(:,kk)/sqrt(V(:,kk)'*M(If,If)*V(:,kk));
        for jj=1:1:size(xn,1)
            u_y_modal(jj,kk)=Phi(2+6*(jj-1),kk);
            u_z_modal(jj,kk)=Phi(3+6*(jj-1),kk);
            theta_x_modal(jj,kk)=Phi(4+6*(jj-1),kk);
        end
        lambda(kk) = D(kk,kk);
        freq(kk) = sqrt(lambda(kk))/(2*pi);
    end


    for kk=1:1:size(V,2)
        figure()
        plot(xn(:,1),u_y_modal(:,kk))
        title("u_y mode " + kk)
        figure()
        plot(xn(:,1),u_z_modal(:,kk))
        title("u_z mode " + kk)
        figure()
        plot(xn(:,1),theta_x_modal(:,kk))
        title("\theta_x mode " + kk)
    end


    % Once (re)computed, save them to a separate data file
    save('beam_matrices.mat','K','M');
    % TIP: Add other potential results that can be reused in other parts
    % (e.g. element's length 'l', elements rotations matrices 'R', etc.)

else

    % Load previously computed results
    load('beam_matrices.mat','K','M');

end

% Perform modal analysis
% ...

% Compute external forces vector
% ...

```

```
% Solve system
% ...

%% POSTPROCESS

% Save data for postprocessing in separate script file (useful when results
% from different runs need to be compared)
save('beam_results.mat');

% Include plot functions

for i=1:1:N_nodes
    u_z(i,1) = u(3+6*(i-1),1);
end
for i=1:1:N_nodes
    theta_x(i,1) = u(4+6*(i-1),1);
end


figure()
plot(xn,u_z)
title('Vertical deflection along the spanwise')
switch(problem_type)
    case 1
        legend('bending test at beam model','Location','northwest')
    case 2
        legend('torsion test at beam model','Location','northwest')
end
xlabel('x (m)')
ylabel('u_z (m)')

figure()
plot(xn,theta_x)
title('Twist angle along the spanwise')
switch(problem_type)
    case 1
        legend('bending test at beam model','Location','northwest')
    case 2
        legend('torsion test at beam model','Location','northwest')
end
xlabel('x (m)')
ylabel('\theta_x (rad)')
```

4.2 MAIN SHELL

```
%% PART I - SHELL MODELLING (TEMPLATE)

% Useful commands to initialize script
clear
close all
clear all

%% DATA

problem_type = 2; %1 for bending. 2 for torsion.
Nm = 6; %number of modes

% Define the problem's data (e.g. dimensions, material properties, etc.)
E = 71e9; %Pa
v_poisson = 0.33;
G = E/(2*(1+v_poisson));
rho = 2300; %kg/m3
```

```
18
19 F = 1; %N
20 T = 1; %Nm
21
22 c = 0.8; %m
23 b = 5.5; %m
24 y0 = 0.3083; %m
25 y1 = 0.176; %m
26 y2 = 0.464; %m
27 h1 = 17.5e-3; %m
28 h2 = 15e-3; %m
29 h3 = 6e-3; %m
30 A = 0.0062; %m2
31 Iy = 8.801e-6; %m4
32 Iz = 8.069e-5; %m4
33 J = 8.949e-5; %m4
34 ky = 0.5457;
35 kz = 0.1767;
36 kt = 0.2920;
37 yc = 0.2954; %m
38
39 %% PREPROCESS
40
41 % Load mesh data
42 load('shell.mat','xn','Tn','Tm','indRoot','indPointA','indPointB','indSpar1','
       indSpar2');
43 % xn : Nodal coordinates
44 %       |x1 y1 z1|
45 %       |x2 y2 z2|
46 %       |  ...   |
47 % Tn : Nodal connectivities [Nelem x 4]
48 % Tm : Material connectivities [Nelem x 1]
49 % indRoot   : Array of indices for root section nodes.
50 % indPointA : Index for node at point A.
51 % indPointB : Index for node at point B.
52 % indSpar1  : Array of indices for front spar centerline nodes.
53 % indSpar2  : Array of indices for rear spar centerline nodes.
54
55 % Define boundary conditions and forces data matrices: Up, Fe, Pe, Be
56 N_elements = size(Tn,1);
57 N_nodes = size(xn,1);
58
59 kk=0;
60 for i=1:1:size(indRoot,1)
61     for j=1:1:6
62         kk=kk+1;
63         Up(kk,1) = 0;
64         Up(kk,2) = indRoot(i,1);
65         Up(kk,3) = j;
66     end
67 end
68
69 Fa_shear = F*((y2-yc)/(y2-y1));
70 Fb_shear = F*((yc-y1)/(y2-y1));
71 Fa_torque = -T/(y2-y1);
72 Fb_torque = T/(y2-y1);
73
74 switch(problem_type)
75     case 1
76         Fe = [Fa_shear indPointA 3;
77               Fb_shear indPointB 3];
78     case 2
79         Fe = [Fa_torque indPointA 3;
```

```
              Fb_torque indPointB 3];
end

Pe = [0 1 2];
Be = [0 1 2];

h_vector = [h1; h2; h3]; %more rows would be added if more values existed
E_vector = [E; E; E]; %more rows would be added if more values existed
v_poisson_vector = [v_poisson; v_poisson; v_poisson]; %more rows would be added if
    more values existed
rho_vector = [rho; rho; rho]; %more rows would be added if more values existed



%% SOLVER

% Obtain system matrices

% TIP: To avoid recomputing the system matrices use a save/load structure:
if 1 % Set to 1 to (re)compute the system matrices and '0' to load them

    % Compute system matrices (as long as parameters don't change there is
    % no need to repeat the matrix assembly on every run)


    %--section 2-----------------------------------
    Ndof = 6*N_nodes;
    K=sparse(Ndof,Ndof);
    M=sparse(Ndof,Ndof);
    %K=zeros(Ndof,Ndof);
    %M=zeros(Ndof,Ndof);

    for e=1:1:N_elements
        term1=xn(Tn(e,3),:)'-xn(Tn(e,1),:)';
        term2=xn(Tn(e,4),:)'-xn(Tn(e,2),:)';
        S = cross(term1,term2)/2;
        k_dir = S/norm(S);
        d = (xn(Tn(e,2),:)'+xn(Tn(e,3),:)'-xn(Tn(e,4),:)'-xn(Tn(e,1),:)')/2;
        i_dir = d/norm(d);
        j_dir = cross(k_dir,i_dir);
        Rp=[i_dir(1,1) j_dir(1,1) k_dir(1,1) 0 0;
            i_dir(2,1) j_dir(2,1) k_dir(2,1) 0 0;
            i_dir(3,1) j_dir(3,1) k_dir(3,1) 0 0;
            0 0 0 i_dir(1,1) j_dir(1,1);
            0 0 0 i_dir(2,1) j_dir(2,1);
            0 0 0 i_dir(3,1) j_dir(3,1)]';

        R(:,:,e)=zeros(20,24);
        R(1:5,1:6,e)=Rp;
        R(6:10,7:12,e)=Rp;
        R(11:15,13:18,e)=Rp;
        R(16:20,19:24,e)=Rp;

        a_coef = [-1 1 1 -1];
        b_coef = [-1 -1 1 1];
        N1=[1 1 1 1]'/4;
        N1_xi = a_coef/4;
        N1_nu = b_coef/4;
        J1 = zeros(2,2);

        for i=1:1:4
            matrix_Ni = [N1_xi(i); N1_nu(i)];
            matrix_ij = [i_dir j_dir];
```

```
            J1 = J1 + matrix_Ni*xn(Tn(e,i),:)*matrix_ij;
        end
        N1x = J1\[N1_xi; N1_nu];
        S1=4*det(J1);

        for i=1:1:4
            Bs_i(:,:,i)=[0 0 N1x(1,i) 0 N1(i);
                         0 0 N1x(2,i) -N1(i) 0];
        end

        Cs = eye(2)*5*h_vector(Tm(e,1),1)*E_vector(Tm(e,1),1)/(12*(1+
    v_poisson_vector(Tm(e,1),1)));
        Bs(:,:,e) = [Bs_i(:,:,1) Bs_i(:,:,2) Bs_i(:,:,3) Bs_i(:,:,4)];
        Ks(:,:,e) = S1*R(:,:,e)'*Bs(:,:,e)'*Cs*Bs(:,:,e)*R(:,:,e);

        for i=1:1:4
            Bmt_i(:,:,i) = [N1x(2,i) N1x(1,i) 0 0 0];
        end
        Cmt = h_vector(Tm(e,1),1)*E_vector(Tm(e,1),1)/(2*(1+v_poisson_vector(Tm(e
    ,1),1)));
        Bmt(:,:,e) = [Bmt_i(:,:,1) Bmt_i(:,:,2) Bmt_i(:,:,3) Bmt_i(:,:,4)];
        Km(:,:,e) = S1*R(:,:,e)'*Bmt(:,:,e)'*Cmt*Bmt(:,:,e)*R(:,:,e);

        Kb(:,:,e) = zeros(24,24);
        Me(:,:,e) = zeros(24,24);
        xi4 = [-1 1 1 -1]/sqrt(3);
        nu4 = [-1 -1 1 1]/sqrt(3);
        w4 = [1 1 1 1];

        for kk=1:1:4
            J4 = zeros(2,2);
            for i=1:1:4
                N4(i)=(1+a_coef(i)*xi4(kk))*(1+b_coef(i)*nu4(kk))/4;
                N4_xi(1,i)=a_coef(i)*(1+b_coef(i)*nu4(kk))/4;
                N4_nu(1,i)=b_coef(i)*(1+a_coef(i)*xi4(kk))/4;
                matrix_N4 = [N4_xi(i); N4_nu(i)];
                J4 = J4 + matrix_N4*xn(Tn(e,i),:)*matrix_ij;
            end
            N4x = J4\[N4_xi; N4_nu];
            S4(e,kk) = w4(kk)*det(J4);
            for i=1:1:4
                Bmn_i(:,:,i)=[N4x(1,i) 0 0 0 0;
                              0 N4x(2,i) 0 0 0];
            end
            Cmn = [1 v_poisson_vector(Tm(e,1),1); v_poisson_vector(Tm(e,1),1) 1]*
    h_vector(Tm(e,1),1)*E_vector(Tm(e,1),1)/(1-v_poisson_vector(Tm(e,1),1)^2);
            Bmn(:,:,e,kk) = [Bmn_i(:,:,1) Bmn_i(:,:,2) Bmn_i(:,:,3) Bmn_i(:,:,4)];
            Km(:,:,e) = Km(:,:,e) + S4(e,kk)*R(:,:,e)'*Bmn(:,:,e,kk)'*Cmn*Bmn(:,:,
    e,kk)*R(:,:,e);
            for i=1:1:4
                Bb_i(:,:,i) = [0 0 0 0 N4x(1,i);
                               0 0 0 N4x(2,i) 0;
                               0 0 0 -N4x(1,i) N4x(2,i)];
            end
            Cb = [1 v_poisson_vector(Tm(e,1),1) 0; v_poisson_vector(Tm(e,1),1) 1
    0; 0 0 (1-v_poisson_vector(Tm(e,1),1))/2]*h_vector(Tm(e,1),1)^3*E_vector(Tm(e
    ,1),1)/(12*(1-v_poisson_vector(Tm(e,1),1)^2));
            Bb(:,:,e,kk) = [Bb_i(:,:,1) Bb_i(:,:,2) Bb_i(:,:,3) Bb_i(:,:,4)];
            Kb(:,:,e) = Kb(:,:,e) + S4(e,kk)*R(:,:,e)'*Bb(:,:,e,kk)'*Cb*Bb(:,:,e,
    kk)*R(:,:,e);
            for i=1:1:4
                N_i(:,:,i) = N4(i)*eye(5);
            end
```

```
198             rho_matrix = rho_vector(Tm(e,1),1)*h_vector(Tm(e,1),1)*[1 0 0 0 0;
199                                                                     0 1 0 0 0;
200                                                                     0 0 1 0 0;
201                                                                     0 0 0 h_vector
    (Tm(e,1),1)^2/12 0;
202                                                                     0 0 0 0
    h_vector(Tm(e,1),1)^2/12];
203             N(:,:,e,kk) = [N_i(:,:,1) N_i(:,:,2) N_i(:,:,3) N_i(:,:,4)];
204             Me(:,:,e) = Me(:,:,e) + S4(e,kk)*R(:,:,e)'*N(:,:,e,kk)'*rho_matrix*N
    (:,:,e,kk)*R(:,:,e);
205         end
206         for jj=1:1:6
207             Idof(jj,1) = 6*(Tn(e,1)-1)+jj;
208             Idof(6+jj,1) = 6*(Tn(e,2)-1)+jj;
209             Idof(12+jj,1) = 6*(Tn(e,3)-1)+jj;
210             Idof(18+jj,1) = 6*(Tn(e,4)-1)+jj;
211         end
212         K(Idof,Idof) = K(Idof,Idof) + Km(:,:,e) + Kb(:,:,e) + Ks(:,:,e);
213         M(Idof,Idof) = M(Idof,Idof) + Me(:,:,e);
214
215     end
216
217     %--section 3---------------------------------------
218     n = zeros(3,N_nodes);
219     for e=1:1:N_elements
220         term11=xn(Tn(e,3),:)'-xn(Tn(e,1),:)';
221         term22=xn(Tn(e,4),:)'-xn(Tn(e,2),:)';
222         SS = cross(term11,term22)/2;
223         SS_mod(e) = sqrt(SS(1)^2+SS(2)^2+SS(3)^2);
224         kp(:,e) = SS/SS_mod(e);
225         for i=1:1:size(Tn,2)
226             n(:,Tn(e,i)) = n(:,Tn(e,i)) + kp(:,e);
227         end
228     end
229
230     %Kr=zeros(Ndof,Ndof);
231     Kr=sparse(Ndof,Ndof);
232     for e=1:1:N_elements
233         for i=1:1:size(Tn,2)
234             alpha = acos(dot(n(:,Tn(e,i)),kp(:,e))/norm(n(:,Tn(e,i))));
235             alpha_deg = alpha*360/(2*pi);
236             if alpha_deg<0.1
237                 Idof = 6*(Tn(e,i)-1)+[4 5 6]';
238                 Kr(Idof,Idof) = Kr(Idof,Idof) + E_vector(Tm(e,1),1)*h_vector(Tm(e
    ,1),1)*SS_mod(e)*kp(:,e)*kp(:,e)';
239             end
240         end
241     end
242
243     K=K+Kr;
244
245
246     %--section 4---------------------------------------
247     f = zeros(Ndof,1);
248     for qq=1:1:size(Fe,1)
249         f(6*(Fe(qq,2)-1)+Fe(qq,3),1)=f(6*(Fe(qq,2)-1)+Fe(qq,3),1) + Fe(qq,1);
250     end
251     P=zeros(N_nodes,6);
252     for rr=1:1:size(Pe,1)
253         P(Pe(rr,2), Pe(rr,3))=P(Pe(rr,2), Pe(rr,3)) + Pe(rr,1);
254     end
255     B=zeros(N_nodes,6);
256     for sss=1:1:size(Be,1)
```

```
        B(Be(sss,2), Be(sss,3))=B(Be(sss,2), Be(sss,3)) + Be(sss,1);
    end

    for e=1:1:N_elements
        bbb(:,e) = [B(Tn(e,1),:) B(Tn(e,2),:) B(Tn(e,3),:) B(Tn(e,4),:)]';
        p(:,e) = [P(Tn(e,1),:) P(Tn(e,2),:) P(Tn(e,3),:) P(Tn(e,4),:)]';
        fe(:,e) = Me(:,:,e)*bbb(:,e);
        for kk=1:1:4
            fe(:,e) = fe(:,e) + S4(e,kk)*R(:,:,e)'*N(:,:,e,kk)'*N(:,:,e,kk)*R(:,:,e)*p(:,e);
        end
        for jj=1:1:6
            Idof(jj,1) = 6*(Tn(e,1)-1)+jj;
            Idof(6+jj,1) = 6*(Tn(e,2)-1)+jj;
            Idof(12+jj,1) = 6*(Tn(e,3)-1)+jj;
            Idof(18+jj,1) = 6*(Tn(e,4)-1)+jj;
        end
        f(Idof,1)=f(Idof,1)+fe(:,e);
    end


    %--section 5---------------------------------------
    u = zeros(Ndof,1);
    for pp=1:1:size(Up,1)
        Ip(pp)=6*(Up(pp,2)-1)+Up(pp,3);
        u(Ip(pp),1) = Up(pp,1);
    end
    If=setdiff(1:Ndof,Ip);


    %--section 6---------------------------------------
    u(If,1)=K(If,If)\(f(If,1)-K(If,Ip)*u(Ip,1));
    fR = K*u-f;


    %--section 7---------------------------------------
    for e=1:1:N_elements
        for jj=1:1:6
            Idof(jj,1) = 6*(Tn(e,1)-1)+jj;
            Idof(6+jj,1) = 6*(Tn(e,2)-1)+jj;
            Idof(12+jj,1) = 6*(Tn(e,3)-1)+jj;
            Idof(18+jj,1) = 6*(Tn(e,4)-1)+jj;
        end
        for kk=1:1:4
            epsilon_b(:,e,kk)=Bb(:,:,e,kk)*R(:,:,e)*u(Idof,1);
            epsilon_m(1:2,e,kk)=Bmn(:,:,e,kk)*R(:,:,e)*u(Idof,1);
            epsilon_m(3,e,kk)=Bmt(:,:,e)*R(:,:,e)*u(Idof,1);
            epsilon_s(:,e,kk)=Bs(:,:,e)*R(:,:,e)*u(Idof,1);
        end
        Cp=[1 v_poisson_vector(Tm(e,1),1) 0; v_poisson_vector(Tm(e,1),1) 1 0; 0 0 (1-v_poisson_vector(Tm(e,1),1))/2]*E_vector(Tm(e,1),1)/(1-v_poisson_vector(Tm(e,1),1)^2);
        Cs=eye(2)*E_vector(Tm(e,1),1)/(2*(1+v_poisson_vector(Tm(e,1),1)));
        for kk=1:1:4
            sigma_m(:,e,kk)=Cp*epsilon_m(:,e,kk);
            sigma_s(:,e,kk)=Cs*epsilon_s(:,e,kk);
            sigma_b(:,e,kk)=Cp*h_vector(Tm(e,1),1)*epsilon_b(:,e,kk)/2;
            sigma_plus=[sigma_m(:,e,kk)+sigma_b(:,e,kk); sigma_s(:,e,kk)]';
            sigma_VM_plus=(sigma_plus(1)^2+sigma_plus(2)^2-sigma_plus(1)*sigma_plus(2)+3*(sigma_plus(3)+sigma_plus(4)+sigma_plus(5)))^(1/2);
            sigma_minus=[sigma_m(:,e,kk)-sigma_b(:,e,kk); sigma_s(:,e,kk)]';
            sigma_VM_minus=(sigma_minus(1)^2+sigma_minus(2)^2-sigma_minus(1)*sigma_minus(2)+3*(sigma_minus(3)+sigma_minus(4)+sigma_minus(5)))^(1/2);
```

```
315             sigma_VM(e,kk)=max(sigma_VM_plus,sigma_VM_minus);
316         end
317     end
318 
319             %--section modal--------------------------------------
320 
321     [V,D] = MAIN_modal(K,M,If,Nm);
322 
323     Phi = zeros(Ndof,Nm);
324     lambda = zeros(1,Nm);
325     for kk=1:1:size(V,2)
326         Phi(If,kk) = V(:,kk)/sqrt(V(:,kk)'*M(If,If)*V(:,kk));
327         for jj=1:1:size(indSpar1,1)
328             u_y_modal(jj,kk) = (Phi(2+6*(indSpar1(jj,1)-1),kk)+Phi(2+6*(indSpar2(
    jj,1)-1),kk))/2;
329             theta_x_modal(jj,kk) = (Phi(3+6*(indSpar2(jj,1)-1),kk) - Phi(3+6*(
    indSpar1(jj,1)-1),kk))/(y2-y1);
330             u_z_modal(jj,kk) = Phi(3+6*(indSpar1(jj,1)-1),kk) + theta_x_modal(jj,
    kk)*(yc-y1);
331         end
332         lambda(kk) = D(kk,kk);
333         freq(kk) = sqrt(lambda(kk))/(2*pi);
334     end
335 
336         for kk=1:1:size(V,2)
337         figure()
338         plot(xn(indSpar1(:),1),u_y_modal(:,kk))
339         title("u_y mode " + kk)
340         figure()
341         plot(xn(indSpar1(:),1),u_z_modal(:,kk))
342         title("u_z mode " + kk)
343         figure()
344         plot(xn(indSpar1(:),1),theta_x_modal(:,kk))
345         title("\theta_x mode " + kk)
346         end
347 
348 
349 
350     % Once (re)computed, save them to a separate data file
351     save('shell_matrices.mat','K','M');
352     % TIP: Add other potential results that can be reused in other parts
353     % (e.g. element's length 'l', elements rotations matrices 'R', etc.)
354 
355 else
356 
357     % Load previously computed results
358     load('shell_matrices.mat','K','M');
359 
360 end
361 
362 % Perform modal analysis
363 % ...
364 
365 % Compute external forces vector
366 % ...
367 
368 % Solve system
369 % ...
370 
371 %% POSTPROCESS
372 
373 % Save data for postprocessing in separate script file (useful when results
374 % from different runs need to be compared)
```

```
375 save('shell_results.mat');
376
377 % Include plot functions
378
379 for i=1:1:size(indSpar1,1)
380     theta_x(i,1) = (u(3+6*(indSpar2(i,1)-1),1) - u(3+6*(indSpar1(i,1)-1),1))/(y2-
        y1);
381     u_z(i,1) = u(3+6*(indSpar1(i,1)-1),1) + theta_x(i,1)*(yc-y1);
382     xn_plot(i,1) = xn(indSpar1(i,1),1);
383 end
384
385
386 figure()
387 plot(xn_plot,u_z)
388 title('Vertical deflection along the spanwise')
389 switch(problem_type)
390     case 1
391         legend('bending test at shell model','Location','northwest')
392     case 2
393         legend('torsion test at shell model','Location','northwest')
394 end
395 xlabel('x (m)')
396 ylabel('u_z (m)')
397
398 figure()
399 plot(xn_plot,theta_x)
400 title('Twist angle along the spanwise')
401 switch(problem_type)
402     case 1
403         legend('bending test at shell model','Location','northwest')
404     case 2
405         legend('torsion test at shell model','Location','northwest')
406 end
407 xlabel('x (m)')
408 ylabel('\theta_x (rad)')
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423 % Additional plot functions useful to visualize 3D model and modes
424 %scale = 1;
425 % plotDeformed('shell',xn,Tn,u,scale);
426 % This function plots the deformed structure:
427 % xn : Nodal coordinates matrix [Nnodes x 3]
428 % Tn : Nodal connectivities matrix [Nelem x 4]
429 % u : Displacements vector obtained from the system solution. It is expected
430 %     to be given as a column vector with dimensions [Ndof x 1].
431 % scale : Scale factor to amplify the displacements (set to appropriate
432 %         number to visualize the deformed structure properly).
433 %imodes = [1,2,3,4,5,6];
434 % plotModes('shell',Phi,freq,imodes)
435 % This function plots the specified modes resulting from a modal analysis
436 % in sets of 9.
```

```
% Phi : Modal displacements matrix in which each column corresponds to the
%       mode shape of the corresponding mode. Expected dimensions [Ndof x Nmodes]
% freq : Natural frequencies array. Expected dimensions [Nmodes x 1]
% imodes : Array selecting which modes to plot. A maximum of 9 modes must
%          can be selected. Example: imodes = [1,2,3,4,5,6,7,8,9] will plot
%          the modes stored in the first 9 columns of Phi / imodes = [1,4,5,10]
%          will plot modes in columns 1, 4, 5 and 10 of Phi.
```

4.3 MAIN WING

```
%% PART II - WING MODELLING

% Useful commands to initialize script
clc
clear
close all

%% SOLUTION CONTROLS

problem_type = 3; % 1 for bending, 2 for torsion, 3 for pressure and weight
solution_type = 2; % 1 for direct method, 2 for model-order reduction solution
Nm = 9; % Number of desired modes to be computed (this is the number that will be
    used to solve the system if model-order reduction solution is requested)
Im = [1,2,3,4,6,7]; %Number of modes to be used for the modal projection

WB = 1; %Set to 1 to include, set to 0 to neglect
ST = 1; %Set to 1 to include, set to 0 to neglect
RB = 1; %Set to 1 to include, set to 0 to neglect
SK = 1; %Set to 1 to include, set to 0 to neglect

computeModes = WB*ST*RB*SK; % Don't compute modes if any is 0

if computeModes == 0 && solution_type == 2
    error('Model-order reduction is only implemented for the solving of the entire
     model, it does not work if parts are neglected.')
end


%% DATA

% Define the problem's data (e.g. dimensions, material properties, etc.)
E = 71e9; %Pa
v_poisson = 0.33; %chan chachachan chachachchanchanchan chan chachachachan chachac
rho = 2300; %kg/m3
G = E/(2*(1+v_poisson));

c = 0.8; %m
b = 5.5; %m
y0 = 0.3083; %m
y1 = 0.176; %m
y2 = 0.464; %m
yc = 0.2954; %m

% Loads
F = 1; %N
T = 1; %Nm

% other

p_inf = 12500; % Pa
alpha = 10*pi/180; % rad
g0 = 9.80665; %m/s^2
```

```
51
52
53
54 %% PREPROCESS
55
56 % Load mesh data
57 load('wing.mat','xn','Tn_st','Tm_st','Tn_wb','Tm_wb','Tn_rb','Tm_rb','Tn_sk','
       Tm_sk','indRoot','indPointA','indPointB','indSpar1','indSpar2','n_u','n_l');
58 % xn : Nodal coordinates
59 %       |x1 y1 z1|
60 %       |x2 y2 z2|
61 %       |  ...   |
62 % Tn_st : Nodal connectivities for beam elements (st: stringers) [Nelem x 2]
63 % Tn_wb, Tn_rb, Tn_sk : Nodal connectivities for shell elements (wb: wingbox, rb:
       ribs, sk: skin) [Nelem x 4]
64 % Tm_st, Tm_wb, Tm_rb, Tm_sk : Material connectivities for the different elements
       [Nelem x 1]
65 % indRoot   : Array of indices for root section nodes.
66 % indPointA : Index for node at point A.
67 % indPointB : Index for node at point B.
68 % indSpar1  : Array of indices for front spar centerline nodes.
69 % indSpar2  : Array of indices for rear spar centerline nodes.
70 % n_u, n_l  : Matrices containing information about the unit normals in the upper
71 %              and lower surfaces, respectively.
72 %       |nx1 ny1 nz1 id1|   First three columns are components of normal vector
73 %       |nx2 ny2 nz2 id2|   Last (fourth) column is the nodal index
74 %       |      ...      |
75
76 % Define boundary conditions and forces data matrices: Up, Fe, Pe, Be
77
78 N_nodes = size(xn,1);
79 Ndof = 6*N_nodes;
80
81 % LOAD PREPROCESSING
82
83 kk=0;
84 for i=1:1:size(indRoot,1)
85     for j=1:1:6
86         kk=kk+1;
87         Up(kk,1) = 0;
88         Up(kk,2) = indRoot(i,1);
89         Up(kk,3) = j;
90     end
91 end
92
93 Fa_shear = F*((y2-yc)/(y2-y1));
94 Fb_shear = F*((yc-y1)/(y2-y1));
95 Fa_torque = -T/(y2-y1);
96 Fb_torque = T/(y2-y1);
97
98
99 % WING BOX
100
101 N_elements_wb = size(Tn_wb,1);
102
103 h1_wb = 17.5e-3; %m
104 h2_wb = 15e-3; %m
105 h3_wb = 6e-3; %m
106 h_vector_wb = [h1_wb; h2_wb; h3_wb]; %more rows would be added if more values
       existed
107
108 E_vector_wb = [E;E;E];%more rows would be added if more values existed
```

```
v_poisson_vector_wb = [v_poisson; v_poisson; v_poisson]; %more rows would be added
    if more values existed
rho_vector_wb = [rho; rho; rho]; %more rows would be added if more values existed

switch(problem_type)
    case 1
        Fe_wb = [Fa_shear indPointA 3;
              Fb_shear indPointB 3];
        Pe_wb = [];
    case 2
        Fe_wb = [Fa_torque indPointA 3;
              Fb_torque indPointB 3];
        Pe_wb = [];
    case 3
        Fe_wb = [];
        Pe_wb = [];

end



% STRINGERS

N_elements_st = size(Tn_st,1);

ky_st = 5/6;
kz_st = 5/6;
kt_st = 1;

r_st = 5*10^-3; %m

A_st = pi*(r_st)^2;%more rows would be added if more values existed


Iy_st = (pi/4)*r_st^4; %m^4
Iz_st = (pi/4)*r_st^4; %m^4

J_st = (pi/2)*r_st^4; %m^4

E_vector_st = E;%more rows would be added if more values existed

G_vector_st = G;%more rows would be added if more values existed

rho_vector_st = rho;%more rows would be added if more values existed

A_vector_st = A_st; %more rows would be added if more values existed

Iy_vector_st = Iy_st;
Iz_vector_st = Iz_st;
J_vector_st = J_st;

ky_vector_st = ky_st;
kz_vector_st = kz_st;
kt_vector_st = kt_st;

j_orientations = [0 1 0];

switch(problem_type)
    case 1
        Fe_st = [];
        Qe_st = [];
    case 2
        Fe_st = [];
```

```
        Qe_st = [];
    case 3
        Fe_st = [];
        Qe_st = [];
end

% RIBS

N_elements_rb = size(Tn_rb,1);

h1_rb = 5e-3; %m

h_vector_rb = [h1_rb]; %more rows would be added if more values existed

E_vector_rb = [E];%more rows would be added if more values existed
v_poisson_vector_rb = [v_poisson]; %more rows would be added if more values
    existed
rho_vector_rb = [rho]; %more rows would be added if more values existed

switch(problem_type)
    case 1
        Fe_rb = [];
        Pe_rb = [];
    case 2
        Fe_rb = [];
        Pe_rb = [];
    case 3
        Fe_rb = [];
        Pe_rb = [];
end

% SKIN

N_elements_sk = size(Tn_sk,1);

h1_sk = 2e-3; %m

h_vector_sk = [h1_sk]; %more rows would be added if more values existed

E_vector_sk = [E];%more rows would be added if more values existed
v_poisson_vector_sk = [v_poisson]; %more rows would be added if more values
    existed
rho_vector_sk = [rho]; %more rows would be added if more values existed

switch(problem_type)
    case 1
        Fe_sk = [];
        Pe_sk = [];
    case 2
        Fe_sk = [];
        Pe_sk = [];
    case 3
        Fe_sk = [];
        Pe_sk = ComputePressureState(xn,n_u,n_l,b,c,p_inf,alpha);
end

% BODY FORCES
switch(problem_type)
    case 1
        Be = [];
    case 2
        Be = [];
    case 3
```

```
        for ii = 1:size(xn,1)

              Be(ii,:) = [-g0 ii 3];

        end

end


%% SOLVER

% Obtain system matrices

% TIP: To avoid recomputing the system matrices use a save/load structure:
if  1% Set to 1 to (re)compute the system matrices and '0' to load them

    % Compute system matrices (as long as parameters don't change there is
    % no need to repeat the matrix assembly on every run)
    % ...
    [K_wb,M_wb,f_wb,Bb_wb,Bmn_wb,Bmt_wb,Bs_wb,R_wb] = ComputeKM_shells(xn,N_nodes,
    N_elements_wb,Tn_wb,Tm_wb,h_vector_wb,E_vector_wb,v_poisson_vector_wb,
    rho_vector_wb,Fe_wb,Be,Pe_wb);
    [K_st,M_st,f_st] = ComputeKM_beams(xn,N_nodes,N_elements_st,Tn_st,Tm_st,
    E_vector_st,G_vector_st,rho_vector_st,j_orientations,A_vector_st,Iy_vector_st,
    Iz_vector_st,J_vector_st,ky_vector_st,kz_vector_st,kt_vector_st,Fe_st,Be,Qe_st)
    ;
    [K_rb,M_rb,f_rb,Bb_rb,Bmn_rb,Bmt_rb,Bs_rb,R_rb] = ComputeKM_shells(xn,N_nodes,
    N_elements_rb,Tn_rb,Tm_rb,h_vector_rb,E_vector_rb,v_poisson_vector_rb,
    rho_vector_rb,Fe_rb,Be,Pe_rb);
    [K_sk,M_sk,f_sk,Bb_sk,Bmn_sk,Bmt_sk,Bs_sk,R_sk] = ComputeKM_shells(xn,N_nodes,
    N_elements_sk,Tn_sk,Tm_sk,h_vector_sk,E_vector_sk,v_poisson_vector_sk,
    rho_vector_sk,Fe_sk,Be,Pe_sk);

    % Once (re)computed, save them to a separate data file
    save('wing_matrices.mat','K_wb','M_wb','K_st','M_st','K_rb','M_rb','K_sk','
    M_sk','f_wb','f_st','f_rb','f_sk');
    save('wing_aux_matrices.mat','Bb_wb','Bmn_wb','Bmt_wb','Bs_wb','R_wb','Bb_rb',
    'Bmn_rb','Bmt_rb','Bs_rb','R_rb','Bb_sk','Bmn_sk','Bmt_sk','Bs_sk','R_sk');
    % TIP: Add other potential results that can be reused in other parts
    % (e.g. element's length 'l', elements rotations matrices 'R', etc.)
else

    % Load previously computed results
    load('wing_matrices.mat','K_wb','M_wb','K_st','M_st','K_rb','M_rb','K_sk','
    M_sk','f_wb','f_st','f_rb','f_sk');
    load('wing_aux_matrices.mat','Bb_wb','Bmn_wb','Bmt_wb','Bs_wb','R_wb','Bb_rb',
    'Bmn_rb','Bmt_rb','Bs_rb','R_rb','Bb_sk','Bmn_sk','Bmt_sk','Bs_sk','R_sk');
end


% Compute global matrices

K = WB*K_wb + ST*K_st + RB*K_rb + SK*K_sk;
M = WB*M_wb + ST*M_st + RB*M_rb + SK*M_sk;

% Compute global force vector

f = f_wb + f_st + f_rb + f_sk;

% Boundary conditions

    u = zeros(Ndof,1);
    for pp=1:1:size(Up,1)
```

```
        Ip(pp)=6*(Up(pp,2)-1)+Up(pp,3);
        u(Ip(pp),1) = Up(pp,1);
    end
    If=setdiff(1:Ndof,Ip);

% Perform modal analysis

if computeModes == 1
[V,D] = MAIN_modal(K,M,If,Nm);

    Phi = zeros(Ndof,Nm);
    lambda = zeros(1,Nm);
    for kk=1:1:size(V,2)
        Phi(If,kk) = V(:,kk)/sqrt(V(:,kk)'*M(If,If)*V(:,kk));
        lambda(kk) = D(kk,kk);
        freq(kk) = sqrt(lambda(kk))/(2*pi);
    end
end

% Solve system

if solution_type == 1 % Direct solution
    u(If,1)=K(If,If)\(f(If,1)-K(If,Ip)*u(Ip,1));
    fR = K*u-f;
elseif solution_type == 2 % Model-order reduction solution
    F = f; % more columns would be added if different frequencies were analyzed.
    In this case, only wk = 0 is analyzed (static).
    omega = 0; % more columns would be added if different frequencies were
    analyzed. In this case, only wk = 0 is analyzed (static).
    u = zeros(Ndof,length(omega));
    for kk = 1:size(F,2)
        for jj = 1:length(Im)
            alpha_modal(jj,kk) = (Phi(:,Im(jj)))'*F(:,kk)/(lambda(jj)-omega(kk)^2)
    ;
            u(:,kk) = u(:,kk) + Phi(:,Im(jj))*alpha_modal(jj,kk);
        end
    end

else
    error('Requested solution type does not exist. The currently implemented
    solution types are: 1 for direct solving; 2 for solving with model-order
    reduction.')
end

% Posterior calculations

sigVM_wb = Compute_stress_shells(N_elements_wb,u,Tn_wb,Tm_wb,h_vector_wb,
    v_poisson_vector_wb,E_vector_wb,Bb_wb,Bmn_wb,Bmt_wb,Bs_wb,R_wb);
sigVM_rb = Compute_stress_shells(N_elements_rb,u,Tn_rb,Tm_rb,h_vector_rb,
    v_poisson_vector_rb,E_vector_rb,Bb_rb,Bmn_rb,Bmt_rb,Bs_rb,R_rb);
sigVM_sk = Compute_stress_shells(N_elements_sk,u,Tn_sk,Tm_sk,h_vector_sk,
    v_poisson_vector_sk,E_vector_sk,Bb_sk,Bmn_sk,Bmt_sk,Bs_sk,R_sk);




%% POSTPROCESS

% Save data for postprocessing in separate script file (useful when results
% from different runs need to be compared)
%save('wing_results.mat');

% Include plot functions
```

```
337
338 for i=1:1:size(indSpar1,1)
339     theta_x(i,1) = (u(3+6*(indSpar2(i,1)-1),1) - u(3+6*(indSpar1(i,1)-1),1))/(y2-
    y1);
340     u_z(i,1) = u(3+6*(indSpar1(i,1)-1),1) + theta_x(i,1)*(yc-y1);
341     xn_plot(i,1) = xn(indSpar1(i,1),1);
342 end
343
344
345 figure()
346 plot(xn_plot,u_z)
347 title('Vertical deflection along the spanwise')
348 switch(problem_type)
349     case 1
350         legend('bending test at wing model','Location','northwest')
351     case 2
352         legend('torsion test at wing model','Location','northwest')
353     case 3
354         legend('wing model under pressure and body forces','Location','northwest')
355 end
356 xlabel('x (m)')
357 ylabel('u_z (m)')
358
359 figure()
360 plot(xn_plot,theta_x)
361 title('Twist angle along the spanwise')
362 switch(problem_type)
363     case 1
364         legend('bending test at wing model','Location','northwest')
365     case 2
366         legend('torsion test at wing model','Location','northwest')
367     case 3
368         legend('wing model under pressure and body forces','Location','northwest')
369 end
370 xlabel('x (m)')
371 ylabel('\theta_x (rad)')
372
373 % Additional plot functions useful to visualize 3D model and modes
374 scale = 1;
375 plotDeformed('wing',xn,Tn_wb,u,scale,sigVM_wb); % For wingbox elements
376 plotDeformed('wing',xn,Tn_rb,u,scale,sigVM_rb); % For rib elements
377 plotDeformed('wing',xn,Tn_sk,u,scale,sigVM_sk); % For skin elements
378
379 % This function plots the deformed structure and Von Mises stress distribution:
380 % xn : Nodal coordinates matrix [Nnodes x 3]
381 % Tn : Nodal connectivities matrix [Nelem x 4]
382 % u : Displacements vector obtained from the system solution. It is expected
383 %     to be given as a column vector with dimensions [Ndof x 1].
384 % scale : Scale factor to amplify the displacements (set to appropriate
385 %         number to visualize the deformed structure properly).
386 % sigVM : Von Mises stress at each Gauss point. It is expected to be given as
387 %         a matrix with dimensions [Nelem x Ngauss].
388 imodes = [1,2,3,4,5,6,7,8,9];
389
390 plotModes('wing',Phi,freq,imodes)
391 % This function plots the specified modes resulting from a modal analysis
392 % in sets of 9.
393 % Phi : Modal displacements matrix in which each column corresponds to the
394 %       mode shape of the corresponding mode. Expected dimensions [Ndof x Nmodes]
395 % freq : Natural frequencies array. Expected dimensions [Nmodes x 1]
396 % imodes : Array selecting which modes to plot. A maximum of 9 modes must
397 %          can be selected. Example: imodes = [1,2,3,4,5,6,7,8,9] will plot
398 %          the modes stored in the first 9 columns of Phi / imodes = [1,4,5,10]
```

```
%           will plot modes in columns 1, 4, 5 and 10 of Phi.
```

4.4 MAIN MODAL

Note that, despite the name, this is a function that computes the eigenvectors, not a main.

```
function [V,D] = MAIN_modal(K,M,If,Nm)
K = (K+K')/2;
M = (M+M')/2;
[V,D] = eigs(K(If,If),M(If,If),Nm,'sm');
end
```

4.5 COMPUTE PRESSURE STATE

```
function Pe = ComputePressureState(xn,n_u,n_l,b,c,p_inf,alpha)

Pe = sparse(3*(size(n_u,1)+size(n_l,1)),3);

kk = 0;

for ii = 1:size(n_u,1)

    node = n_u(ii,4);
    x = xn(node,1);
    y = xn(node,2);

    if x <= 0.8*b

        Px = p_inf;

    else

        Px = p_inf*cos((pi/2)*(x-0.8*b)/(0.2*b));

    end

    pu = -4*alpha*Px*((1-y/c)^4+sqrt(1-y/c));

    P = -pu*n_u(ii,1:3);

    kk = kk+1;
    Pe(kk,:) = [P(1) node 1];
    kk = kk+1;
    Pe(kk,:) = [P(2) node 2];
    kk = kk+1;
    Pe(kk,:) = [P(3) node 3];
end

for ii = 1:size(n_l,1)

    node = n_l(ii,4);
    x = xn(node,1);
    y = xn(node,2);

    if x <= 0.8*b

        Px = p_inf;

    else

        Px = p_inf*cos((pi/2)*(x-0.8*b)/(0.2*b));
```

```
48
49      end
50
51      pl = 4*alpha*Px*((1-y/c)^4-0.25*sqrt(1-y/c));
52
53      P = -pl*n_l(ii,1:3);
54
55      kk = kk+1;
56      Pe(kk,:) = [P(1) node 1];
57      kk = kk+1;
58      Pe(kk,:) = [P(2) node 2];
59      kk = kk+1;
60      Pe(kk,:) = [P(3) node 3];
61  end
62
63  end
```

4.6 COMPUTE KM BEAMS

```
1   function [K,M,f] = ComputeKM_beams(xn,N_nodes,N_elements,Tn,Tm,E_vector,G_vector,
        rho_vector,j_orientations,A_vector,Iy_vector,Iz_vector,J_vector,ky_vector,
        kz_vector,kt_vector,Fe,Be,Qe)
2
3       % Compute system matrices (as long as parameters don't change there is
4       % no need to repeat the matrix assembly on every run)
5
6       %--section 2----------------------------------------
7       Ndof = 6*N_nodes;
8       K=sparse(Ndof,Ndof);
9       M=sparse(Ndof,Ndof);
10
11      for e=1:1:N_elements
12
13          l(e) = norm(xn(Tn(e,2),:)-xn(Tn(e,1),:));
14          i = ((xn(Tn(e,2),:))' - (xn(Tn(e,1),:))')/l(e);
15          j = j_orientations(Tm(e,1),:)';
16          k = cross(i,j);
17          Rp = [i(1,1) j(1,1) k(1,1) 0 0 0;
18                i(2,1) j(2,1) k(2,1) 0 0 0;
19                i(3,1) j(3,1) k(3,1) 0 0 0;
20                0 0 0 i(1,1) j(1,1) k(1,1);
21                0 0 0 i(2,1) j(2,1) k(2,1);
22                0 0 0 i(3,1) j(3,1) k(3,1)]';
23
24          R(:,:,e) = [Rp(1,1) Rp(1,2) Rp(1,3) Rp(1,4) Rp(1,5) Rp(1,6) 0 0 0 0 0 0;
25                      Rp(2,1) Rp(2,2) Rp(2,3) Rp(2,4) Rp(2,5) Rp(2,6) 0 0 0 0 0 0;
26                      Rp(3,1) Rp(3,2) Rp(3,3) Rp(3,4) Rp(3,5) Rp(3,6) 0 0 0 0 0 0;
27                      Rp(4,1) Rp(4,2) Rp(4,3) Rp(4,4) Rp(4,5) Rp(4,6) 0 0 0 0 0 0;
28                      Rp(5,1) Rp(5,2) Rp(5,3) Rp(5,4) Rp(5,5) Rp(5,6) 0 0 0 0 0 0;
29                      Rp(6,1) Rp(6,2) Rp(6,3) Rp(6,4) Rp(6,5) Rp(6,6) 0 0 0 0 0 0;
30                      0 0 0 0 0 0 Rp(1,1) Rp(1,2) Rp(1,3) Rp(1,4) Rp(1,5) Rp(1,6);
31                      0 0 0 0 0 0 Rp(2,1) Rp(2,2) Rp(2,3) Rp(2,4) Rp(2,5) Rp(2,6);
32                      0 0 0 0 0 0 Rp(3,1) Rp(3,2) Rp(3,3) Rp(3,4) Rp(3,5) Rp(3,6);
33                      0 0 0 0 0 0 Rp(4,1) Rp(4,2) Rp(4,3) Rp(4,4) Rp(4,5) Rp(4,6);
34                      0 0 0 0 0 0 Rp(5,1) Rp(1,2) Rp(5,3) Rp(5,4) Rp(5,5) Rp(5,6);
35                      0 0 0 0 0 0 Rp(6,1) Rp(6,2) Rp(6,3) Rp(6,4) Rp(6,5) Rp(6,6)];
36
37          Nx(1)=-1/l(e);
38          Nx(2)=1/l(e);
39          Ba(1,:,e) = [Nx(1) 0 0 0 0 0 Nx(2) 0 0 0 0 0];
40          Ca = E_vector(Tm(e,1),1)*A_vector(Tm(e,1),1);
41          Ka(:,:,e) = l(e)*R(:,:,e)'*Ba(1,:,e)'*Ca*Ba(1,:,e)*R(:,:,e);
```

```
42
43          Bb(:,:,e) = [0 0 0 0 Nx(1) 0 0 0 0 0 Nx(2) 0;
44                       0 0 0 0 0 Nx(1) 0 0 0 0 0 Nx(2)];
45
46          Cb = E_vector(Tm(e,1),1)*[Iy_vector(Tm(e,1),1) 0;
47                                    0 Iz_vector(Tm(e,1),1)];
48
49          Kb(:,:,e) = l(e)*R(:,:,e)'*Bb(:,:,e)'*Cb*Bb(:,:,e)*R(:,:,e);
50
51          N_Gauss = 1/2;
52
53          Bs(:,:,e) = [0 Nx(1) 0 0 0 -N_Gauss 0 Nx(2) 0 0 0 -N_Gauss;
54                       0 0 Nx(1) 0 N_Gauss 0 0 0 Nx(2) 0 N_Gauss 0];
55
56          Cs = G_vector(Tm(e,1),1)*A_vector(Tm(e,1),1)*[ky_vector(Tm(e,1),1) 0;
57                                                        0 kz_vector(Tm(e,1),1)];
58
59          Ks(:,:,e)=l(e)*R(:,:,e)'*Bs(:,:,e)'*Cs*Bs(:,:,e)*R(:,:,e);
60
61          Bt(1,:,e) = [0 0 0 Nx(1) 0 0 0 0 0 Nx(2) 0 0];
62          Ct = G_vector(Tm(e,1),1)*J_vector(Tm(e,1),1)*kt_vector(Tm(e,1),1);
63          Kt(:,:,e) = l(e)*R(:,:,e)'*Bt(1,:,e)'*Ct*Bt(1,:,e)*R(:,:,e);
64
65          Xi = [-1/sqrt(3);
66                  1/sqrt(3)];
67          w = [1;
68               1];
69
70          rho_matrix = rho_vector(Tm(e,1),1)*[A_vector(Tm(e,1),1) 0 0 0 0 0;
71                                              0 A_vector(Tm(e,1),1) 0 0 0 0;
72                                              0 0 A_vector(Tm(e,1),1) 0 0 0;
73                                              0 0 0 J_vector(Tm(e,1),1) 0 0;
74                                              0 0 0 0 Iy_vector(Tm(e,1),1) 0;
75                                              0 0 0 0 0 Iz_vector(Tm(e,1),1)];
76          Me(:,:,e)=zeros(12,12);
77
78          for kk=1:1:2
79              N(1) = (1-Xi(kk))/2;
80              N(2) = (1+Xi(kk))/2;
81              N_matrix(:,:,e,kk)=[N(1) 0 0 0 0 0 N(2) 0 0 0 0 0;
82                                  0 N(1) 0 0 0 0 0 N(2) 0 0 0 0;
83                                  0 0 N(1) 0 0 0 0 0 N(2) 0 0 0;
84                                  0 0 0 N(1) 0 0 0 0 0 N(2) 0 0;
85                                  0 0 0 0 N(1) 0 0 0 0 0 N(2) 0;
86                                  0 0 0 0 0 N(1) 0 0 0 0 0 N(2)];
87              Me(:,:,e) = Me(:,:,e) + (w(kk)*l(e)*R(:,:,e)'*N_matrix(:,:,e,kk)'*
    rho_matrix*N_matrix(:,:,e,kk)*R(:,:,e))/2;
88
89          end
90
91          for jj=1:1:6
92              Idof(jj,1)=6*(Tn(e,1)-1)+jj;
93              Idof(6+jj,1)=6*(Tn(e,2)-1)+jj;
94          end
95
96          K(Idof,Idof)=K(Idof,Idof) + Ka(:,:,e) + Kb(:,:,e) + Ks(:,:,e) + Kt(:,:,e);
97          M(Idof,Idof)=M(Idof,Idof) + Me(:,:,e);
98
99      end
100
101     %--section 3-------------------------------------
102         f=zeros(Ndof,1);
103         for q=1:1:size(Fe,1)
```

```
            f(6*(Fe(q,2)-1)+Fe(q,3),1) = f(6*(Fe(q,2)-1)+Fe(q,3),1) + Fe(q,1);
        end

        Q = zeros(N_nodes,6);

        for r=1:1:size(Qe,1)
            Q(Qe(r,2),Qe(r,3)) = Q(Qe(r,2),Qe(r,3)) + Qe(r,1);
        end

        B = zeros(N_nodes,6);

        for s=1:1:size(Be,1)
            B(Be(s,2),Be(s,3)) = B(Be(s,2),Be(s,3)) + Be(s,1);
        end

        for e=1:1:N_elements
            bb(:,e) = [B(Tn(e,1),:),B(Tn(e,2),:)]';
            qq(:,e) = [Q(Tn(e,1),:),Q(Tn(e,2),:)]';
            fe(:,e) = Me(:,:,e)*bb(:,e);
            for kk=1:1:2
                fe(:,e) = fe(:,e) + (w(kk)*l(e)*R(:,:,e)'*N_matrix(:,:,e,kk)'*
    N_matrix(:,:,e,kk)*R(:,:,e)*qq(:,e))/2;
            end

            for jj=1:1:6
                Idof(jj,1) = 6*(Tn(e,1)-1)+jj;
                Idof(6+jj,1) = 6*(Tn(e,2)-1)+jj;
            end

            f(Idof,1) = f(Idof,1) + fe(:,e);
        end

end
```

4.7 COMPUTE KM SHELLS

```
function [K,M,f,Bb,Bmn,Bmt,Bs,R] = ComputeKM_shells(xn,N_nodes,N_elements,Tn,Tm,
    h_vector,E_vector,v_poisson_vector,rho_vector,Fe,Be,Pe)

%--section 2--------------------------------------
    Ndof = 6*N_nodes;
    K=sparse(Ndof,Ndof);
    M=sparse(Ndof,Ndof);

    for e=1:1:N_elements
        term1=xn(Tn(e,3),:)'-xn(Tn(e,1),:)';
        term2=xn(Tn(e,4),:)'-xn(Tn(e,2),:)';
        S = cross(term1,term2)/2;
        k_dir = S/norm(S);
        d = (xn(Tn(e,2),:)'+xn(Tn(e,3),:)'-xn(Tn(e,4),:)'-xn(Tn(e,1),:)')/2;
        i_dir = d/norm(d);
        j_dir = cross(k_dir,i_dir);
        Rp=[i_dir(1,1) j_dir(1,1) k_dir(1,1) 0 0;
            i_dir(2,1) j_dir(2,1) k_dir(2,1) 0 0;
            i_dir(3,1) j_dir(3,1) k_dir(3,1) 0 0;
            0 0 0 i_dir(1,1) j_dir(1,1);
            0 0 0 i_dir(2,1) j_dir(2,1);
            0 0 0 i_dir(3,1) j_dir(3,1)]';

        R(:,:,e)=zeros(20,24);
        R(1:5,1:6,e)=Rp;
        R(6:10,7:12,e)=Rp;
```

```
26          R(11:15,13:18,e)=Rp;
27          R(16:20,19:24,e)=Rp;
28
29          a_coef = [-1 1 1 -1];
30          b_coef = [-1 -1 1 1];
31          N1=[1 1 1 1]'/4;
32          N1_xi = a_coef/4;
33          N1_nu = b_coef/4;
34          J1 = zeros(2,2);
35
36          for i=1:1:4
37              matrix_Ni = [N1_xi(i); N1_nu(i)];
38              matrix_ij = [i_dir j_dir];
39              J1 = J1 + matrix_Ni*xn(Tn(e,i),:)*matrix_ij;
40          end
41          N1x = J1\[N1_xi; N1_nu];
42          S1=4*det(J1);
43
44          for i=1:1:4
45              Bs_i(:,:,i)=[0 0 N1x(1,i) 0 N1(i);
46                          0 0 N1x(2,i) -N1(i) 0];
47          end
48
49          Cs = eye(2)*5*h_vector(Tm(e,1),1)*E_vector(Tm(e,1),1)/(12*(1+
    v_poisson_vector(Tm(e,1),1)));
50          Bs(:,:,e) = [Bs_i(:,:,1) Bs_i(:,:,2) Bs_i(:,:,3) Bs_i(:,:,4)];
51          Ks(:,:,e) = S1*R(:,:,e)'*Bs(:,:,e)'*Cs*Bs(:,:,e)*R(:,:,e);
52
53          for i=1:1:4
54              Bmt_i(:,:,i) = [N1x(2,i) N1x(1,i) 0 0 0];
55          end
56          Cmt = h_vector(Tm(e,1),1)*E_vector(Tm(e,1),1)/(2*(1+v_poisson_vector(Tm(e
    ,1),1)));
57          Bmt(:,:,e) = [Bmt_i(:,:,1) Bmt_i(:,:,2) Bmt_i(:,:,3) Bmt_i(:,:,4)];
58          Km(:,:,e) = S1*R(:,:,e)'*Bmt(:,:,e)'*Cmt*Bmt(:,:,e)*R(:,:,e);
59
60          Kb(:,:,e) = zeros(24,24);
61          Me(:,:,e) = zeros(24,24);
62          xi4 = [-1 1 1 -1]/sqrt(3);
63          nu4 = [-1 -1 1 1]/sqrt(3);
64          w4 = [1 1 1 1];
65
66          for kk=1:1:4
67              J4 = zeros(2,2);
68              for i=1:1:4
69                  N4(i)=(1+a_coef(i)*xi4(kk))*(1+b_coef(i)*nu4(kk))/4;
70                  N4_xi(1,i)=a_coef(i)*(1+b_coef(i)*nu4(kk))/4;
71                  N4_nu(1,i)=b_coef(i)*(1+a_coef(i)*xi4(kk))/4;
72                  matrix_N4 = [N4_xi(i); N4_nu(i)];
73                  J4 = J4 + matrix_N4*xn(Tn(e,i),:)*matrix_ij;
74              end
75              N4x = J4\[N4_xi; N4_nu];
76              S4(e,kk) = w4(kk)*det(J4);
77              for i=1:1:4
78                  Bmn_i(:,:,i)=[N4x(1,i) 0 0 0 0;
79                                0 N4x(2,i) 0 0 0];
80              end
81              Cmn = [1 v_poisson_vector(Tm(e,1),1); v_poisson_vector(Tm(e,1),1) 1]*
    h_vector(Tm(e,1),1)*E_vector(Tm(e,1),1)/(1-v_poisson_vector(Tm(e,1),1)^2);
82              Bmn(:,:,e,kk) = [Bmn_i(:,:,1) Bmn_i(:,:,2) Bmn_i(:,:,3) Bmn_i(:,:,4)];
83              Km(:,:,e) = Km(:,:,e) + S4(e,kk)*R(:,:,e)'*Bmn(:,:,e,kk)'*Cmn*Bmn(:,:,
    e,kk)*R(:,:,e);
84              for i=1:1:4
```

```
                Bb_i(:,:,i) = [0 0 0 0 N4x(1,i);
                               0 0 0 N4x(2,i) 0;
                               0 0 0 -N4x(1,i) N4x(2,i)];
            end
            Cb = [1 v_poisson_vector(Tm(e,1),1) 0; v_poisson_vector(Tm(e,1),1) 1
0; 0 0 (1-v_poisson_vector(Tm(e,1),1))/2]*h_vector(Tm(e,1),1)^3*E_vector(Tm(e
,1),1)/(12*(1-v_poisson_vector(Tm(e,1),1)^2));
            Bb(:,:,e,kk) = [Bb_i(:,:,1) Bb_i(:,:,2) Bb_i(:,:,3) Bb_i(:,:,4)];
            Kb(:,:,e) = Kb(:,:,e) + S4(e,kk)*R(:,:,e)'*Bb(:,:,e,kk)'*Cb*Bb(:,:,e,
kk)*R(:,:,e);
            for i=1:1:4
                N_i(:,:,i) = N4(i)*eye(5);
            end
            rho_matrix = rho_vector(Tm(e,1),1)*h_vector(Tm(e,1),1)*[1 0 0 0 0;
                                                                    0 1 0 0 0;
                                                                    0 0 1 0 0;
                                                                    0 0 0 h_vector
(Tm(e,1),1)^2/12 0;
                                                                    0 0 0 0
h_vector(Tm(e,1),1)^2/12];
            N(:,:,e,kk) = [N_i(:,:,1) N_i(:,:,2) N_i(:,:,3) N_i(:,:,4)];
            Me(:,:,e) = Me(:,:,e) + S4(e,kk)*R(:,:,e)'*N(:,:,e,kk)'*rho_matrix*N
(:,:,e,kk)*R(:,:,e);
        end
        for jj=1:1:6
            Idof(jj,1) = 6*(Tn(e,1)-1)+jj;
            Idof(6+jj,1) = 6*(Tn(e,2)-1)+jj;
            Idof(12+jj,1) = 6*(Tn(e,3)-1)+jj;
            Idof(18+jj,1) = 6*(Tn(e,4)-1)+jj;
        end
        K(Idof,Idof) = K(Idof,Idof) + Km(:,:,e) + Kb(:,:,e) + Ks(:,:,e);
        M(Idof,Idof) = M(Idof,Idof) + Me(:,:,e);

    end

    %--section 3---------------------------------------
    n = zeros(3,N_nodes);
    for e=1:1:N_elements
        term11=xn(Tn(e,3),:)'-xn(Tn(e,1),:)';
        term22=xn(Tn(e,4),:)'-xn(Tn(e,2),:)';
        SS = cross(term11,term22)/2;
        SS_mod(e) = sqrt(SS(1)^2+SS(2)^2+SS(3)^2);
        kp(:,e) = SS/SS_mod(e);
        for i=1:1:size(Tn,2)
            n(:,Tn(e,i)) = n(:,Tn(e,i)) + kp(:,e);
        end
    end

    %Kr=zeros(Ndof,Ndof);
    Kr=sparse(Ndof,Ndof);
    for e=1:1:N_elements
        for i=1:1:size(Tn,2)
            alpha = acos(dot(n(:,Tn(e,i)),kp(:,e))/norm(n(:,Tn(e,i))));
            alpha_deg = alpha*360/(2*pi);
            if alpha_deg<0.1
                Idof = 6*(Tn(e,i)-1)+[4 5 6]';
                Kr(Idof,Idof) = Kr(Idof,Idof) + E_vector(Tm(e,1),1)*h_vector(Tm(e
,1),1)*SS_mod(e)*kp(:,e)*kp(:,e)';
            end
        end
    end

    K=K+Kr;
```

```
141
142
143 % Compute external forces vector
144
145 %--section 4--------------------------------------
146     f = zeros(Ndof,1);
147     for qq=1:1:size(Fe,1)
148         f(6*(Fe(qq,2)-1)+Fe(qq,3),1)=f(6*(Fe(qq,2)-1)+Fe(qq,3),1) + Fe(qq,1);
149     end
150     P=zeros(N_nodes,6);
151     for rr=1:1:size(Pe,1)
152         P(Pe(rr,2), Pe(rr,3))=P(Pe(rr,2), Pe(rr,3)) + Pe(rr,1);
153     end
154     B=zeros(N_nodes,6);
155     for sss=1:1:size(Be,1)
156         B(Be(sss,2), Be(sss,3))=B(Be(sss,2), Be(sss,3)) + Be(sss,1);
157     end
158
159     for e=1:1:N_elements
160         bbb(:,e) = [B(Tn(e,1),:) B(Tn(e,2),:) B(Tn(e,3),:) B(Tn(e,4),:)]';
161         p(:,e) = [P(Tn(e,1),:) P(Tn(e,2),:) P(Tn(e,3),:) P(Tn(e,4),:)]';
162         fe(:,e) = Me(:,:,e)*bbb(:,e);
163         for kk=1:1:4
164             fe(:,e) = fe(:,e) + S4(e,kk)*R(:,:,e)'*N(:,:,e,kk)'*N(:,:,e,kk)*R(:,:,
    e)*p(:,e);
165         end
166         for jj=1:1:6
167             Idof(jj,1) = 6*(Tn(e,1)-1)+jj;
168             Idof(6+jj,1) = 6*(Tn(e,2)-1)+jj;
169             Idof(12+jj,1) = 6*(Tn(e,3)-1)+jj;
170             Idof(18+jj,1) = 6*(Tn(e,4)-1)+jj;
171         end
172         f(Idof,1)=f(Idof,1)+fe(:,e);
173     end
```

4.8 COMPUTE STRESS SHELLS

```
1  function sigma_VM = Compute_stress_shells(N_elements,u,Tn,Tm,h_vector,
       v_poisson_vector,E_vector,Bb,Bmn,Bmt,Bs,R)
2  for e=1:1:N_elements
3          for jj=1:1:6
4              Idof(jj,1) = 6*(Tn(e,1)-1)+jj;
5              Idof(6+jj,1) = 6*(Tn(e,2)-1)+jj;
6              Idof(12+jj,1) = 6*(Tn(e,3)-1)+jj;
7              Idof(18+jj,1) = 6*(Tn(e,4)-1)+jj;
8          end
9          for kk=1:1:4
10             epsilon_b(:,e,kk)=Bb(:,:,e,kk)*R(:,:,e)*u(Idof,1);
11             epsilon_m(1:2,e,kk)=Bmn(:,:,e,kk)*R(:,:,e)*u(Idof,1);
12             epsilon_m(3,e,kk)=Bmt(:,:,e)*R(:,:,e)*u(Idof,1);
13             epsilon_s(:,e,kk)=Bs(:,:,e)*R(:,:,e)*u(Idof,1);
14         end
15         Cp=[1 v_poisson_vector(Tm(e,1),1) 0; v_poisson_vector(Tm(e,1),1) 1 0; 0 0
       (1-v_poisson_vector(Tm(e,1),1))/2]*E_vector(Tm(e,1),1)/(1-v_poisson_vector(Tm(e
       ,1),1)^2);
16         Cs=eye(2)*E_vector(Tm(e,1),1)/(2*(1+v_poisson_vector(Tm(e,1),1)));
17         for kk=1:1:4
18             sigma_m(:,e,kk)=Cp*epsilon_m(:,e,kk);
19             sigma_s(:,e,kk)=Cs*epsilon_s(:,e,kk);
20             sigma_b(:,e,kk)=Cp*h_vector(Tm(e,1),1)*epsilon_b(:,e,kk)/2;
21             sigma_plus=[sigma_m(:,e,kk)+sigma_b(:,e,kk); sigma_s(:,e,kk)]';
```

```
22              sigma_VM_plus=(sigma_plus(1)^2+sigma_plus(2)^2-sigma_plus(1)*
    sigma_plus(2)+3*(sigma_plus(3)+sigma_plus(4)+sigma_plus(5)))^(1/2);
23              sigma_minus=[sigma_m(:,e,kk)-sigma_b(:,e,kk); sigma_s(:,e,kk)]';
24              sigma_VM_minus=(sigma_minus(1)^2+sigma_minus(2)^2-sigma_minus(1)*
    sigma_minus(2)+3*(sigma_minus(3)+sigma_minus(4)+sigma_minus(5)))^(1/2);
25              sigma_VM(e,kk)=max(sigma_VM_plus,sigma_VM_minus);
26          end
27      end
28  end
```